Mini Project Report

on

ProblySUS

Submitted by

Midhun M (20223070)
Muhammed Faaz Razi (20223072)
Muhammed Irfan E (20223073)
Muhammed Sahal (20223074)
Nidal CMP (20223077)

In partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering.

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

APRIL 2026

DIVISION OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

CERTIFICATE

Certified that this is a Mini Project Report titled

problySUS

Submitted by

Midhun M (20223070)
Muhammed Faaz Razi (20223072)
Muhammed Irfan E (20223073)
Muhammed Sahal (20223074)
Nidal CMP (20223077)

of VI Semester, Computer Science and Engineering in the year 2024 in partial fulfillment requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering of Cochin University of Science and Technology.

Mr. Pramod Pavithran	Ms. Sheena S	Ms. Amrutha S Nair
Head of Division	**Project Coordinator**	**Project Guide**

ACKNOWLEDGEMENT

First and foremost, we offer our deepest gratitude and humble prayers to the Almighty, whose grace and blessings have been our guiding light throughout this project. We express our heartfelt appreciation to our esteemed Project Guide, Ms. Amrutha S Nair, for her invaluable guidance, unwavering patience, and continued support throughout the development of ProblySUS, her insightful feedback and encouragement were instrumental in shaping the successful implementation of this project. We also extend our sincere thanks to our Project Coordinator, Ms. Sheena S, for her constructive guidance and constant support during every phase of our work, and to Mr. Pramod Pavithran, Head of the Department, for his leadership and direction that kept our project on the right path. We remain truly grateful to our friends and classmates for their helpful suggestions, cooperation, and motivation throughout the development process, as their involvement played a vital role in refining our ideas and execution. Last but not least, we would like to express our profound gratitude to our families for their endless encouragement, understanding, and moral support, without which this accomplishment would not have been possible.

Midhun M (20223070)
Muhammed Faaz Razi (20223072)
Muhammed Irfan E (20223073)
Muhammed Sahal (20223074)
Nidal CMP (20223077)

DECLARATION

We, Midhun M, Muhammed Faaz Razi, Muhammed Irfan E, Muhammed Sahal, and Nidal CMP, hereby declare that the project titled **ProblySUS** is a bona fide work carried out by us during the year 2026, under the guidance of **Ms. Amrutha S Nair**, Assistant Professor, School of Engineering, CUSAT. We further declare that this project has not been submitted, either in part or full, for the award of any degree, diploma, associateship, fellowship, or any other academic recognition in any university or institution.

ABSTRACT

ProblySUS is a comprehensive web-based security system designed to detect scam websites, phishing pages, and fraudulent online platforms using a multi-signal analysis approach. This project addresses the limitations of traditional detection methods by combining multiple independent indicators such as domain intelligence, DNS analysis, threat intelligence feeds, content inspection, and real-time browser behavior analysis to provide a reliable and explainable risk assessment of websites. The system integrates advanced techniques including headless browser simulation using Playwright to capture dynamic website behavior such as redirects and external network requests, which are not detectable through static analysis alone. Additionally, ProblySUS incorporates privacy auditing and tracker identification to evaluate website data collection practices. Built using Python and Flask for the backend, and React with Tailwind CSS for the frontend, the platform also includes a browser extension for real-time URL scanning within the user's browsing environment. The system generates a composite risk score ranging from 0 to 100 and classifies websites into Safe, Caution, Suspicious, or Fraudulent categories, along with clear explanations for each decision. With features such as parallel processing, local threat feed integration, and a fast analysis mode, ProblySUS provides a scalable and efficient solution for enhancing user awareness and improving online safety. **Keywords:** Scam Detection, Phishing Detection, Website Security, Threat Intelligence, DNS Analysis, Headless Browser, Privacy Auditing, Risk Scoring, Web Application, Browser Extension

List of Figures

4.1 Overall System Architecture Flowchart of ProblySUS 15
4.2 Data Flow Diagram of ProblySUS 16
4.3 Use Case Diagram of ProblySUS 17

5.1 Home Page / URL Input Screen 23
5.2 Analysis Results Dashboard — Score Gauge and Risk Label 23
5.3 Behavior Panel — Redirects, External Requests, Suspicious Domains 24
5.4 Network Panel — Domain Classification and Risk Level 24
5.5 Privacy Panel — Exposure Bar, Fingerprinting Indicators, Cookie Counts . . . 25
5.6 Tracker Panel — Named Tracker Detection 25
5.7 app.py Route Structure and Parallel Execution 26
5.8 scorer.py Weighted Penalty Calculation 27
5.9 validator.py and pattern_checker.py Implementation 28
5.10 behavior_checker.py Playwright Intercept Logic 29
5.11 privacy_checker.py Cookie and Fingerprinting Detection 30
5.12 URLhaus Feed Download and CSV Extraction 31
5.13 Browser Extension Popup — Overview Tab 32
5.14 Browser Extension Popup — Behavior and Privacy Tabs 33
5.15 Fast Mode and Result Cache Logic in app.py 34

6.1 Web Application Home Page — URL Input Interface 36
6.2 Analysis Result Dashboard with Animated Panels 36
6.3 Flask app.py Route Definitions and ThreadPoolExecutor Block 38
6.4 Playwright Launch and Network Interception Logic 39
6.5 Chrome Extension Popup — Overview Tab 40
6.6 Chrome Extension Popup — Behavior and Privacy Tabs 41
6.7 URLhaus Feed Download and Auto-Refresh Logic 42
6.8 Global Error Handling and Input Validation Logic in app.py 44

List of Tables

4.1 Result Cache Structure 18
4.2 Blacklist Data Structure 18
4.3 Signal Weights in the ProblySUS Scoring Model 18

6.1 Backend Analysis Libraries 37

7.1 Unit Testing Results for ProblySUS Modules 46
7.2 Performance Testing Results 47
7.3 Bug Fixes and Enhancements 48

Contents

Acknowledgement i

Declaration ii

Abstract iii

List of Figures iv

List of Tables iv

1 Introduction 1
- 1.1 Introduction . . . 1
  - 1.1.1 Methodology . . . 1
  - 1.1.2 Features . . . 2
  - 1.1.3 Significance of Work . . . 3

2 Literature Review 5
- 2.1 URL-Based Phishing Detection Using Lexical Features . . . 5
- 2.2 Domain Registration and WHOIS Intelligence in Fraud Detection . . . 5
- 2.3 Threat Intelligence Feeds and Blacklist-Based Detection . . . 6
- 2.4 Content and Behavioral Signals in Phishing Page Detection . . . 6
- 2.5 Network Footprint Analysis and Privacy Profiling of Websites . . . 7

3 System Study 8
- 3.1 Existing System . . . 8
- 3.2 Proposed System . . . 8
- 3.3 Software Requirements Specification . . . 9
  - 3.3.1 Overall Description . . . 9
  - 3.3.2 Product Functions . . . 10
- 3.4 Requirements Specification . . . 11
  - 3.4.1 External Interfaces . . . 11
  - 3.4.2 Functional Requirements . . . 12
  - 3.4.3 Non-Functional Requirements . . . 13

4 System Design 14
- 4.1 Overall System Architecture Flowchart . . . 14
- 4.2 Data Flow Diagram (DFD) . . . 15
- 4.3 Use Case Diagram . . . 16
- 4.4 Data Design . . . 17
  - 4.4.1 Result Cache . . . 17

4.4.2 Blacklist Index . . . 17
4.5 Scoring Model and Signal Weights . . . 18
4.5.1 Compound Penalties . . . 18
4.5.2 Risk Labels . . . 19
4.6 Input-Output Design . . . 19
4.6.1 URL Submission . . . 19
4.6.2 Validation and Static Analysis Output . . . 19
4.6.3 Deep Analysis Output . . . 19
4.6.4 Browser Extension Output . . . 20
4.7 Modular Design . . . 20

5 System Implementation **22**
5.1 Modular Implementation . . . 22
5.1.1 Frontend Development (React + Vite + Tailwind CSS v4) . . . 22
5.1.2 Backend Development (Python / Flask) . . . 26
5.1.3 Analysis Module Implementation . . . 28
5.1.4 Threat Intelligence Data Layer . . . 29
5.1.5 Browser Extension Implementation . . . 32
5.1.6 Fast Mode and Caching . . . 33

6 Technologies, Stacks and Tools Used **35**
6.1 Development Environment . . . 35
6.2 Frontend Technologies . . . 35
6.2.1 React 19 + Vite 7 . . . 35
6.2.2 Tailwind CSS v4 . . . 35
6.2.3 Framer Motion . . . 36
6.2.4 Axios . . . 37
6.3 Backend Technologies . . . 37
6.3.1 Python 3 / Flask . . . 37
6.3.2 Concurrent Execution — ThreadPoolExecutor . . . 37
6.3.3 Playwright + Chromium . . . 37
6.3.4 Analysis Libraries . . . 37
6.4 Browser Extension Technologies . . . 40
6.5 Threat Intelligence Tools . . . 40
6.5.1 URLhaus Feed . . . 40
6.5.2 Local Tracker Database . . . 41
6.6 Testing Tools . . . 43
6.7 Version Control and Project Management . . . 43
6.8 Deployment . . . 43
6.9 Security Considerations . . . 43

7 Testing **45**
7.1 Unit Testing 45
7.2 Integration Testing 45
7.3 Performance Testing 46
7.4 Usability Testing 47
7.5 Bug Fixes and Improvements 47

Chapter 1

Introduction

1.1 Introduction

In today's increasingly digital world, the internet has become an indispensable resource for communication, commerce, and information. However, this widespread connectivity has also given rise to a growing threat landscape of scam websites, phishing pages, and fraudulent online platforms that deceive and exploit unsuspecting users. Traditional approaches to identifying such threats — such as manually checking URLs or relying on browser warnings — are often insufficient, inconsistent, and inaccessible to non-technical users.

To address these limitations, we propose **ProblySUS**, a full-stack, multi-signal scam and phishing detection system that analyzes website URLs and produces a comprehensive, explainable risk assessment. The system integrates multiple techniques including domain intelligence, DNS analysis, threat feed lookups, content inspection, headless browser simulation, privacy auditing, and tracker identification to classify websites as **Safe, Caution, Suspicious, or Fraudulent** through a composite risk score ranging from 0 to 100.

By combining these signals into a single, accessible platform — available both as a web application and a browser extension — ProblySUS serves as an intelligent, real-time security companion for everyday internet users.

1.1.1 Methodology

The development of ProblySUS followed a structured and iterative methodology, beginning with a thorough requirement analysis. To understand the nature of the problem, we studied common phishing and scam patterns, reviewed existing detection tools, and examined open threat intelligence resources. This revealed that no single signal is sufficient for reliable scam detection — effective identification requires the combination of multiple independent indicators across URL structure, domain metadata, network behavior, and page content.

Based on these insights, we proceeded to technology stack selection. The backend was developed using the **Python / Flask** framework, chosen for its simplicity, flexibility, and rich ecosystem of analysis libraries. For domain analysis, we integrated `python-whois` and `dnspython`; for content inspection, `beautifulsoup4` and `textstat`; and for real browser simulation, **Playwright with headless Chromium**, enabling the interception of JavaScript-driven redirects and runtime network requests that static HTTP analysis cannot detect.

The frontend was built using **React 19 with Vite 7** and styled with **Tailwind CSS v4**, with **Framer Motion** for animations and **Axios** for API communication. Additionally, a **Chrome browser extension (Manifest V3)** was developed to bring one-click scanning directly into the user's browsing context.

In the system architecture design phase, we adopted a **modular pipeline approach**, wherein each analysis concern — URL validation, domain age, DNS records, URL patterns, content trust, blacklist lookup, browser behavior, network profiling, privacy grading, and tracker identification — is handled by a dedicated independent module. All module outputs are aggregated and passed to a central risk scoring engine (`scorer.py`) that applies a confidence-weighted penalty system with compound signal amplification and hard overrides for critical threats.

The architecture also incorporates a **ThreadPoolExecutor-based parallel execution model** that significantly reduces analysis time by running independent checks concurrently, and an **in-memory result cache with a 30-minute TTL** to avoid redundant scans.

The development and integration process was iterative. The backend API, frontend panels, and browser extension were developed and tested in parallel, with RESTful JSON serving as the integration contract. Once development was complete, we entered the testing and evaluation phase, which included unit testing of individual modules, integration testing of the full analysis pipeline, and performance testing to validate responsiveness under different network conditions.

A **Fast Mode** (`?fast=true`) was also introduced to allow analysis in **3–5 seconds** by skipping the Playwright browser simulation, compared to the full **8–12 second deep scan**.

1.1.2 Features

ProblySUS is a multi-layered, cross-platform URL security analysis system designed to make scam detection accessible, transparent, and reliable for all users. The system performs over ten independent checks on any submitted URL and presents results through an intuitive, visually rich dashboard.

The system's threat intelligence backbone cross-references submitted domains against a local copy of the **URLhaus threat feed** — a community-maintained database of malicious URLs — that is automatically refreshed every 24 hours without manual intervention. A blacklist hit triggers an immediate **Fraudulent verdict**, bypassing further analysis.

For domains not found on the blacklist, the system evaluates **domain age via WHOIS lookup**, flags newly registered domains (under 30 days) as high-risk phishing signals, and inspects **DNS records** for the presence of MX and SPF entries that legitimate businesses are expected to maintain.

URL pattern analysis detects **IP-based hostnames, abusive top-level domains (.tk, .xyz, .pw), excessive hyphens, and embedded phishing keywords** — all common hallmarks of fraudulent sites. Content analysis fetches the live page and scans for urgency language, sensitive data requests, and the absence of trust indicators such as **Privacy Policy** and **Contact pages**.

Going beyond static analysis, ProblySUS launches a **headless Chromium browser via Playwright** to simulate a real user visit, intercepting all HTTP responses to detect redirect chains and all outgoing requests to profile the site's external network footprint. A dedicated **network intelligence module** classifies each contacted domain as safe infrastructure (CDN/cloud), suspicious, or unknown.

A **privacy auditing module** measures cookie tracking intensity, third-party script loading, and browser fingerprinting techniques (Canvas, WebGL, AudioContext, WebRTC, etc.), assigning a privacy grade from **Good** to **Invasive**. Finally, a **tracker identification module** matches loaded resources against a local database to identify named commercial trackers such as **Google Analytics, Facebook Pixel, and Hotjar**.

All findings are surfaced through a detailed **results dashboard** with animated panels, a risk score gauge, color-coded labels, and an explainable reasons list — as well as through the browser extension's compact popup, which presents the same data in a tabbed interface directly within the browser.

1.1.3 Significance of Work

ProblySUS addresses a critical and growing need in the digital age: empowering everyday users to make informed trust decisions about the websites they visit. With online scams, phishing attacks, and fraudulent e-commerce sites becoming increasingly sophisticated, traditional single-signal detection methods — such as SSL badge checks or basic blacklist lookups — are no longer adequate.

This project delivers a significant advancement by combining ten independent analysis signals into a unified, explainable risk score, reducing both false positives and false negatives through compound signal weighting.

The use of a **headless browser simulation** is particularly significant, as it enables the detection of JavaScript-driven redirects and dynamically loaded malicious resources that no static HTTP-based scanner can observe. The integration of **privacy and tracker auditing** further extends the system's utility beyond pure scam detection, providing users with a holistic view of a site's data collection practices.

The system's **local-first architecture** — with no third-party API keys required and a locally cached threat feed — ensures consistent performance, full privacy of submitted URLs, and zero

dependency on paid external services.

The key contributions of ProblySUS include:

- **Multi-signal composite scoring** — 10+ independent analysis modules aggregated into a single 0–100 risk score with explainable reasons.
- **Runtime behavior analysis** — Playwright headless browser captures JavaScript redirects and asynchronous network requests invisible to static analyzers.
- **Privacy and tracker auditing** — measures fingerprinting intensity and identifies named commercial trackers, providing transparency beyond basic scam detection.
- **Browser extension** — brings one-click URL scanning into the active browsing session with a tabbed detail view and deep-link to the web dashboard.

Chapter 2

Literature Review

The detection of malicious, scam, and phishing websites has been an active area of research for over two decades, evolving alongside the sophistication of online threats. Researchers have explored a wide range of signals — URL structure, domain metadata, page content, network behavior, and threat intelligence feeds — to build reliable detection systems. The following studies highlight the core ideas, methodologies, and technologies that informed and shaped the development of ProblySUS.

2.1 URL-Based Phishing Detection Using Lexical Features

Early research in phishing detection established that a URL's lexical structure alone carries significant predictive power. Studies demonstrated that features such as the use of raw IP addresses as hostnames, excessive hyphens in domain names, the presence of brand names embedded in subdomains, the use of free or abused top-level domains (e.g. `.tk`, `.ml`, `.xyz`), and phishing-related keywords in the URL path (e.g. `login`, `verify`, `paypal`, `secure`) are strong and consistent indicators of fraudulent intent. Machine learning classifiers trained on these features achieved high detection accuracy while remaining computationally lightweight and requiring no page access.

This directly informs ProblySUS's `pattern_checker` module, which performs zero-network, instant lexical inspection of every submitted URL before any heavier analysis begins. By flagging IP-based hostnames, abused TLDs, hyphen counts, and keyword patterns, the module acts as a fast pre-filter that can assign risk points within milliseconds, forming the foundation of the composite scoring pipeline.

2.2 Domain Registration and WHOIS Intelligence in Fraud Detection

Research has consistently identified domain age as one of the strongest individual signals for phishing and scam detection. Fraudulent websites are typically registered shortly before being used in an attack campaign, with the majority of phishing domains being abandoned within 30 to 90 days of registration. Studies leveraging WHOIS registration data showed that newly registered domains (under 30 days old) carry a disproportionately high likelihood of being associated with malicious activity, and that registrar patterns, registration country, and WHOIS

privacy shielding also correlate with fraud risk.

ProblySUS's `whois_checker` module directly applies this insight by querying the WHOIS registry for each submitted domain's creation date and computing its age in days. Domains under one week old are assigned the maximum domain-age penalty, those under one month receive a high penalty, and those under three months receive a moderate penalty — thresholds calibrated from the literature to minimize false positives against legitimately new but benign sites.

2.3 Threat Intelligence Feeds and Blacklist-Based Detection

Community-maintained threat intelligence feeds represent one of the most reliable and widely adopted mechanisms for malicious URL detection. Research evaluating platforms such as URLhaus, PhishTank, and Google Safe Browsing demonstrated that blacklist-based detection achieves near-perfect precision for known threats, with the primary limitation being coverage of newly emerging URLs that have not yet been reported. Studies recommended combining blacklist lookups with complementary heuristic signals to bridge this gap, producing a hybrid detection approach that handles both known and zero-day threats.

ProblySUS integrates URLhaus — a community-maintained, freely available threat feed — as a hard-override detection layer. The `blacklist_checker` and `updater` modules download the URLhaus ZIP feed, extract the CSV, and maintain a local indexed copy that is auto-refreshed every 24 hours. A confirmed blacklist hit returns an instant score of 100 (Fraudulent), bypassing all other analysis, while the remaining modules handle threats not yet listed in the feed.

2.4 Content and Behavioral Signals in Phishing Page Detection

Studies on content-based phishing detection highlighted that fraudulent pages consistently exhibit a distinct set of behavioral and structural patterns: urgency and pressure language ("Act now", "Your account will be suspended"), absence of legitimate trust pages (Privacy Policy, Contact, Terms of Service), requests for sensitive credentials or payment information over unencrypted connections, and unusually high external link ratios. Complementary research on JavaScript-driven attacks demonstrated that many modern phishing pages use client-side redirects, dynamic resource loading, and obfuscated scripts to evade static analysis — underlining the need for runtime browser-based inspection.

These findings motivated two distinct modules in ProblySUS. The `content_checker` module fetches live page HTML and scans for urgency phrases, sensitive data forms, trust indicators, and reading complexity. The `behavior_checker` module addresses the JavaScript evasion challenge by launching a headless Chromium browser via Playwright, which executes

JavaScript exactly as a real browser would — intercepting all HTTP responses to detect redirect chains and capturing all outgoing network requests to identify suspicious runtime domains that no static scan could observe.

2.5 Network Footprint Analysis and Privacy Profiling of Websites

Recent research extended phishing and fraud detection beyond the primary domain to examine a site's broader network footprint — the external services, CDN providers, advertising networks, and analytics platforms it contacts. Studies found that fraudulent sites disproportionately contact domains with suspicious TLDs, IP-based hosts, and excessive numbers of external dependencies, while legitimate sites typically rely on a small set of well-known infrastructure providers (Cloudflare, AWS, Google, Fastly). Parallel work on web privacy identified browser fingerprinting — the use of Canvas, WebGL, AudioContext, and WebRTC APIs to uniquely identify visitors — as a growing surveillance technique employed on a wide range of sites, including fraudulent ones.

ProblySUS operationalizes these findings through two dedicated modules. The `network_checker` module classifies every domain contacted during the Playwright session into `safe_infra`, `suspicious`, or `unknown` buckets and computes an overall network risk level (Low/Medium/High). The `privacy_checker` module independently measures cookie tracking intensity, third-party script count, and the presence of fingerprinting API calls, assigning a privacy grade from Good to Invasive — providing users with a holistic view of a site's data collection practices beyond its direct fraud risk.

Together, these research directions form the theoretical and technical foundation of ProblySUS, collectively motivating its multi-signal, layered approach to scam detection. The combination of lexical URL analysis, WHOIS domain intelligence, community threat feeds, live content and behavior inspection, and network privacy profiling produces a detection system that is both comprehensive in coverage and explainable in output — addressing the core limitations of any single-signal approach and providing everyday users with a transparent, actionable verdict on the trustworthiness of any website.

Chapter 3

System Study

The current landscape of online security tools offers limited, fragmented support for everyday users who need to quickly assess the trustworthiness of a website. Most existing solutions either rely on a single signal (such as a blacklist check or SSL badge) or require technical knowledge to interpret. Through research and observation, we identified the need for a unified, multi-signal platform that is accessible to non-technical users while being thorough enough to catch sophisticated threats. The ProblySUS system study outlines the limitations of existing tools and how our proposed system addresses them.

3.1 Existing System

Most current URL safety tools — such as Google Safe Browsing, VirusTotal URL scanner, or built-in browser warnings — provide basic threat detection. However, these platforms:

- Rely primarily on blacklist lookups and miss zero-day phishing sites not yet reported.
- Do not perform live behavioral analysis of how a page actually behaves in a browser.
- Provide no insight into domain age, DNS infrastructure gaps, or content-level trust indicators.
- Offer no privacy or tracker auditing — users receive no information about how aggressively a site profiles them.
- Present results as simple pass/fail flags with no explainable reasoning, making it difficult for users to understand why a site is flagged.

These systems function as reactive lookup tools rather than proactive, intelligent assessors of website trustworthiness.

3.2 Proposed System

ProblySUS is an end-to-end, multi-signal scam and phishing detection platform that analyzes any submitted URL through ten independent analysis modules and produces a composite risk

score (0–100) with a label — *Safe*, *Caution*, *Suspicious*, or *Fraudulent* — backed by a human-readable list of reasons.

Core Advantages:

- Multi-signal analysis combining domain intelligence, URL patterns, DNS records, live content, blacklist lookup, and headless browser simulation.
- Runtime behavioral analysis via Playwright that detects JavaScript-driven redirects and dynamic external requests invisible to static scanners.
- Privacy and tracker auditing revealing cookie tracking intensity, fingerprinting techniques, and named commercial trackers.
- Fully explainable output — every score point is justified with a plain-language reason.
- Accessible through both a web application and a Chrome browser extension for seamless in-browser scanning.
- Local-first architecture requiring no third-party API keys, with an auto-refreshing URLhaus threat feed.

These functionalities are modular, enabling independent maintenance and future extension of any analysis component without affecting the rest of the pipeline.

3.3 Software Requirements Specification

ProblySUS is designed to empower everyday internet users with an intelligent, transparent tool for assessing website safety. The system combines heuristic, behavioral, and content-based signals into a unified risk assessment. The application is intended for a broad audience including students, professionals, and non-technical users who frequently encounter unfamiliar URLs.

The system accepts a URL as input and generates a comprehensive risk report including a numeric score, risk label, ordered list of reasons, SSL/HTTPS status, domain age, DNS infrastructure details, blacklist status, browser behavior summary, network domain classification, privacy grade, and identified trackers. The platform is accessible via a React-based web application and a Chrome browser extension, both connected to the same Flask REST API backend.

3.3.1 Overall Description

ProblySUS operates as a self-contained platform with a decoupled frontend and backend communicating through a REST API. The backend runs locally on port 5000 while the frontend

operates on port 5173, with CORS enabled for cross-origin communication. The Chrome extension connects directly to the local backend, allowing both interfaces to use the same analysis engine.

The system maintains a local URLhaus threat intelligence database which automatically refreshes every 24 hours via a background thread. This ensures that the threat database remains current without requiring manual updates. Future versions are expected to support cloud deployment for public access and integration with browser-level safe browsing APIs for real-time protection.

3.3.2 Product Functions

The system includes several core functionalities that together provide a comprehensive risk assessment for any submitted URL.

URL Validation and Normalization

Every submitted URL is validated for syntactic correctness. Missing schemes such as `http://` or `https://` are automatically added. The system also checks HTTPS usage and TLS certificate validity before continuing further analysis.

Domain Intelligence

WHOIS registry data is queried to determine the domain age in days. Domains registered within the past 30 days are flagged as high-risk signals. DNS records including MX and SPF are also checked, since the absence of email infrastructure often indicates non-legitimate domains.

URL Pattern Analysis

The URL structure is inspected for suspicious characteristics such as IP-based hostnames, abused top-level domains (e.g., `.tk`, `.xyz`, `.pw`), excessive hyphens, and phishing keywords.

Content Trust Analysis

The system fetches live page HTML and scans it for urgency phrases, sensitive data request forms, presence of trust pages (Privacy Policy, Contact, About), external link ratios, and reading complexity scores.

Blacklist Lookup

The submitted domain is compared against the locally cached URLhaus threat feed. If a match is found, the system immediately returns a *Fraudulent* verdict with a score of 100.

Behavioral Analysis

A headless Chromium browser launched via Playwright loads the target page, executes JavaScript, and captures redirect chains and outgoing network requests. This allows the system to detect behavior that static scanners cannot observe.

Network, Privacy, and Tracker Profiling

External domains contacted during page execution are categorized into safe infrastructure, suspicious, or unknown groups. Cookie usage, third-party scripts, and browser fingerprinting techniques are analyzed. Known commercial trackers are identified using a local tracker database.

Risk Scoring

Outputs from all modules are processed by `scorer.py`, which applies a confidence-weighted penalty model and compound signal amplification to generate the final score and explanatory reasons.

3.4 Requirements Specification

This section describes the functional and non-functional requirements of ProblySUS including system interfaces, performance expectations, and architectural design.

3.4.1 External Interfaces

User Interface

The web application includes a full-page analysis dashboard with a URL input field, animated risk score gauge, color-coded label badge, expandable reasons list, and detailed panels for behavior, network activity, privacy analysis, and tracker detection.

The Chrome extension provides a lightweight popup interface displaying the score arc, key metrics (SSL, domain age, blacklist, MX), risk label, and tabbed views for detailed information.

API Interface

The frontend communicates with the Flask backend through REST APIs.

- `POST /analyze` — accepts a URL and returns the full analysis report in JSON format.
- `POST /api/blacklist/update` — manually triggers an update of the URLhaus blacklist.

Optional parameters include `?fast=true` to skip Playwright analysis and `?force=true` to bypass cached results.

Hardware Requirements

The system requires a machine capable of running Python 3 and Node.js. The Playwright module requires approximately 170 MB for the Chromium browser binary and an active internet connection. A minimum of 4 GB RAM is recommended.

Software Compatibility

The web application supports modern browsers including Chrome, Firefox, Edge, and Safari. The extension supports Chromium-based browsers such as Chrome, Edge, and Brave.

The backend requires Python 3.8+ and dependencies including Flask, flask-cors, requests, python-whois, beautifulsoup4, tldextract, dnspython, textstat, and playwright.

3.4.2 Functional Requirements

- The system shall accept syntactically valid URLs and normalize missing schemes.
- The system shall validate HTTPS usage and TLS certificate integrity.
- The system shall retrieve WHOIS data and classify domain age into risk tiers.
- The system shall check DNS records for MX and SPF entries.
- The system shall perform static URL pattern analysis.
- The system shall analyze live page HTML for trust and phishing indicators.
- The system shall check the domain against the URLhaus blacklist.
- The system shall execute JavaScript using a headless browser for behavioral analysis.
- The system shall analyze external network requests and classify domains.
- The system shall identify commercial trackers and fingerprinting techniques.
- The system shall compute a final composite risk score and assign a risk label.
- The system shall cache analysis results for 30 minutes.
- The system shall update the URLhaus threat feed automatically every 24 hours.

3.4.3 Non-Functional Requirements

Performance

Standard URL analysis should complete within 5–6 seconds. Full analysis including browser simulation should complete within 8–12 seconds. Fast mode analysis should complete within 3–5 seconds. Cached results should be returned instantly.

Usability

The system is designed for non-technical users. Color-coded labels (green, yellow, orange, red), visual indicators, and plain-language explanations ensure that results are easy to interpret.

Reliability

If the Playwright module fails or is unavailable, the system gracefully degrades while continuing other analyses. All modules implement exception handling to prevent system crashes.

Maintainability

Each analysis function is implemented as an independent module following the single-responsibility principle. New signals can be integrated by adding a new module and extending the scoring logic.

Scalability

The system supports parallel execution of analysis modules using a `ThreadPoolExecutor`. Result caching reduces redundant processing. The architecture supports future deployment using containerization technologies such as Docker.

Portability

The system runs on Windows, macOS, and Linux without modification. The frontend is a React/Vite application deployable to any static hosting service, while the backend is a standard Flask application.

Chapter 4

System Design

ProblySUS employs a modular client-server architecture designed for scalability, maintainability, and efficient data exchange. The client layer accepts a URL as input either typed into the web application or automatically read from the active browser tab via the Chrome extension and submits it to the backend analysis pipeline, ensuring a fast and accessible experience across devices.

Main Components

The system consists of three primary layers: Input, Processing, and Output.

The **Input Layer** includes the web frontend URL input bar and the browser extension popup, both capable of accepting any syntactically valid URL.

The **Processing Layer** consists of the Flask backend orchestrating ten analysis modules covering domain intelligence, DNS checks, URL pattern analysis, content inspection, blacklist lookup, headless browser simulation, network classification, privacy auditing, and tracker identification. All module outputs are aggregated by a central scoring engine.

The **Output Layer** delivers a structured JSON risk report rendered as an interactive results dashboard containing a score gauge, risk label, reasons list, and detailed analysis panels.

On the server side, the system manages URL validation, parallel execution of analysis modules, threat intelligence lookups, result caching, and auto-updating of the local blacklist feed. All outputs are returned to the client as a single structured JSON response consumed by both the web frontend and the browser extension. The architecture requires no user authentication since no personal data is stored. Only URL scan results are temporarily cached in memory for 30 minutes.

4.1 Overall System Architecture Flowchart

Figure 4.1 provides a high-level flowchart of the complete ProblySUS workflow from URL submission to final risk presentation. It shows how user input is validated, distributed to parallel analysis modules, merged by the scoring engine, and then returned to both client interfaces.

This flowchart clarifies the control flow and data flow across all major components. It

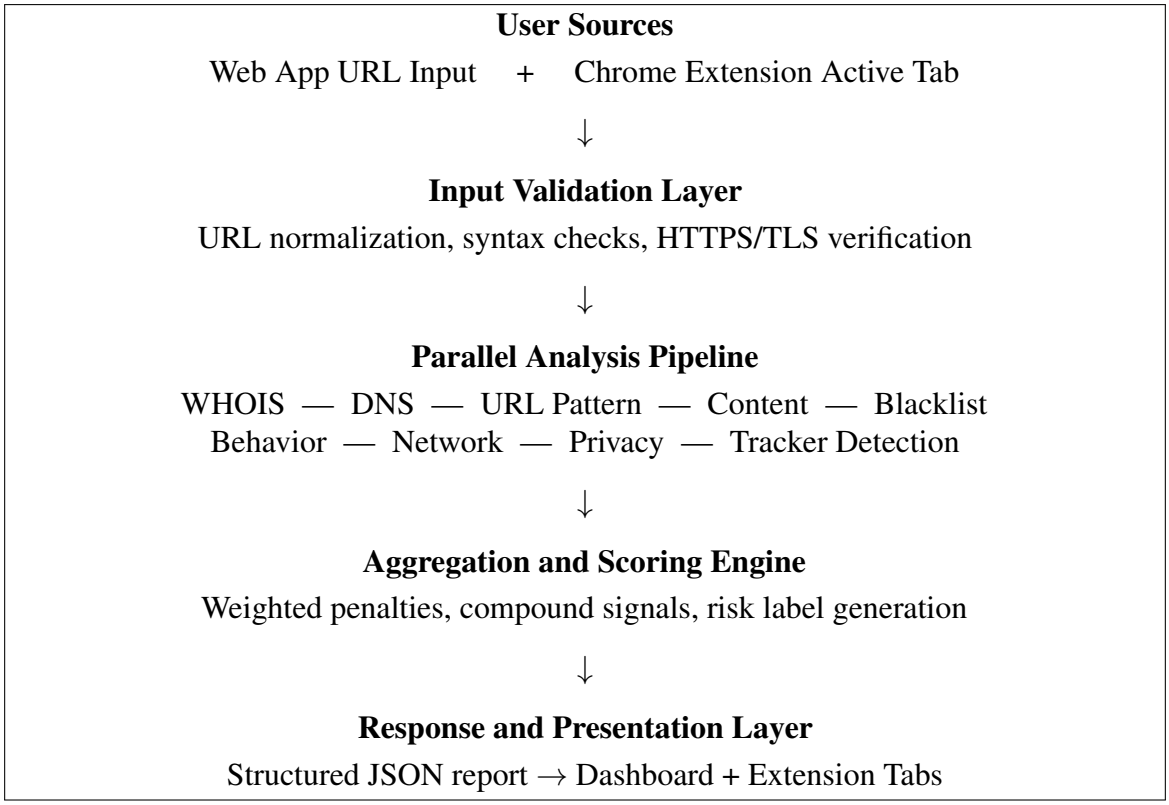

Figure 4.1: Overall System Architecture Flowchart of ProblySUS

highlights that ProblySUS does not rely on a monolithic scan step; instead, it runs specialized modules in parallel and fuses their outputs into one explainable and user-friendly result.

4.2 Data Flow Diagram (DFD)

The Data Flow Diagram illustrates how data moves from the user to the analysis modules and finally to the generated risk report. When a user submits a URL through the web application or browser extension, the system validates the URL and distributes it across multiple analysis modules operating in parallel.

Each module independently analyzes a particular signal domain and returns structured results. These results are then aggregated by the scoring engine to compute the final composite risk score and generate a labeled report returned to the client interface.

The following figure illustrates the operational data path of a scan request in detail. It emphasizes how the same input URL is propagated to multiple analysis modules, then consolidated into a single report, ensuring both analytical depth and consistent output formatting.

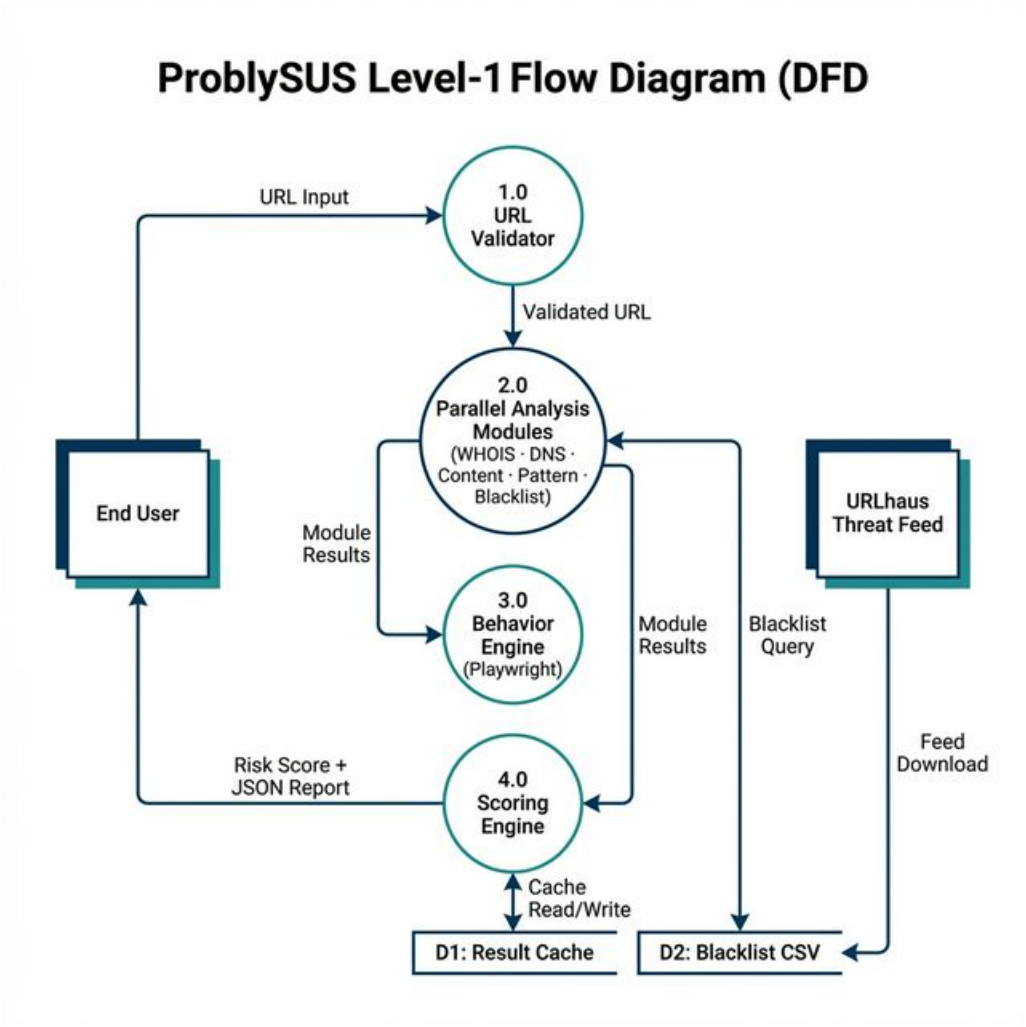

Figure 4.2: Data Flow Diagram of ProblySUS

4.3 Use Case Diagram

The Use Case Diagram defines interactions between users and the system.

ProblySUS has a single primary user role called the **End User**. The end user can perform the following actions:

- Submit a URL for analysis
- View the risk report including score, label, and explanation
- Inspect behavior, network, privacy, and tracker panels
- Trigger manual blacklist update
- Open the full web report from the browser extension

Additionally, the system includes a background component known as the **System Daemon**.

This component automatically refreshes the blacklist feed every 24 hours without requiring user interaction.

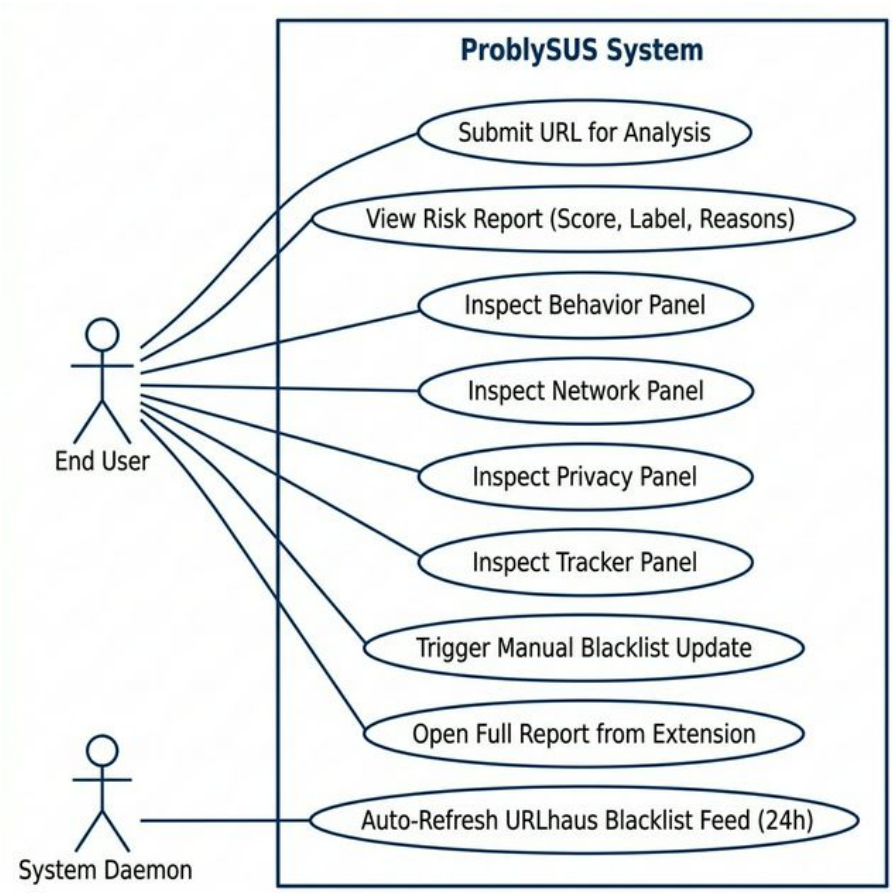

Figure 4.3: Use Case Diagram of ProblySUS

This use case view clarifies user-facing functionality and background automation responsibilities. It helps distinguish actions explicitly initiated by the end user from periodic system tasks such as blacklist refresh, which improves reliability without increasing user effort.

4.4 Data Design

ProblySUS is stateless by design and does not rely on a persistent database. Instead, the system maintains temporary in-memory data structures.

4.4.1 Result Cache

4.4.2 Blacklist Index

The blacklist is loaded from `backend/data/blacklist.csv` into an in-memory dictionary during system startup, enabling constant-time domain lookups.

Property	Description	Type
url	Normalized URL used as cache key	String
result	Full JSON analysis result	Object
expiry	Timestamp of cache expiration (30 min TTL)	DateTime

Table 4.1: Result Cache Structure

Property	Description	Type
domain	Normalized hostname	String
category	Threat category (phishing, malware, etc.)	String
risk_level	High or Critical	Enum
source	Source feed name (URLhaus)	String
last_updated	Timestamp of last feed refresh	DateTime

Table 4.2: Blacklist Data Structure

A tracker signature database stored in `backend/data/trackers.json` maps tracker domains to human-readable names and is loaded during initialization.

4.5 Scoring Model and Signal Weights

The scoring engine implemented in `scorer.py` serves as the analytical core of ProblySUS. It applies a confidence-weighted penalty model to aggregated module outputs and incorporates compound amplification when multiple high-risk signals appear simultaneously.

Signal Category	Max Penalty	Rationale
Blacklist	100 (override)	Confirmed threat
Domain Age	20	Newly registered domains often used in phishing
URL Patterns	18	Suspicious URL characteristics
Behavior	15–18	Runtime redirects and suspicious requests
Content Analysis	15	Urgency language or suspicious forms
HTTPS / SSL	10	Lack of encryption risk
Malicious Scripts	10	Suspicious JavaScript activity
Network Intelligence	10	Suspicious external servers
DNS / MX Records	8	Missing email infrastructure
Tracker Detection	2	Privacy concern indicator
Privacy Signals	2	Fingerprinting or tracking cookies

Table 4.3: Signal Weights in the ProblySUS Scoring Model

4.5.1 Compound Penalties

- New domain (less than 30 days) + aggressive redirects $\rightarrow$ +5

- New domain + suspicious external servers $\rightarrow$ +5
- No HTTPS + sensitive forms + urgency language $\rightarrow$ +5

4.5.2 Risk Labels

- Safe: score < 20
- Caution: score 20–49
- Suspicious: score 50–79
- Fraudulent: score ≥ 80

4.6 Input-Output Design

4.6.1 URL Submission

Users submit URLs through the web application's input bar or the browser extension popup. The system normalizes the URL by adding `https://` if missing and validates the syntax before dispatching it to the backend via `POST /analyze`. The browser extension automatically prefills the current tab's URL.

4.6.2 Validation and Static Analysis Output

The system first performs static checks including HTTPS/TLS validation, URL pattern inspection, and blacklist lookup. These results are displayed immediately as color-coded status indicators such as HTTPS status, domain age, blacklist result, and MX record availability.

4.6.3 Deep Analysis Output

After static checks, detailed results are generated by the analysis modules and displayed through multiple interface panels:

Overview Panel

Displays the composite risk score, risk label, recommendation, and explanation list.

Behavior Panel

Shows redirect count, external requests, suspicious domains, redirect chain sequence, and final page title.

Network Panel

Displays external domain counts, safe infrastructure domains, suspicious domains, and network risk level.

Privacy Panel

Shows privacy grade, exposure score, cookie statistics, third-party scripts, and fingerprinting indicators.

Tracker Panel

Lists detected trackers including tracker name, domain, and associated HTML element.

4.6.4 Browser Extension Output

The browser extension provides a compact tabbed interface displaying the same analysis results. The Overview tab shows the score arc and risk label. Additional tabs present behavior, network, tracker, and privacy information. A button allows users to open the full report in the web application.

4.7 Modular Design

The modular architecture ensures maintainability and extensibility.

Validator Module (`validator.py`)

Acts as the first pipeline stage, validating URLs and checking TLS status.

Domain Intelligence Modules

- `whois_checker.py`
- `dns_checker.py`

These modules retrieve domain age and DNS infrastructure information.

Static Analysis Modules

- `pattern_checker.py`

- `content_checker.py`
- `blacklist_checker.py`

These modules perform static URL inspection, HTML analysis, and blacklist verification.

Runtime Analysis Modules

- `behavior_checker.py`
- `network_checker.py`
- `privacy_checker.py`
- `tracker_checker.py`

These modules execute runtime browser analysis, classify network activity, analyze privacy exposure, and detect trackers.

Scoring Engine (`scorer.py`)

Aggregates outputs from all modules and computes the final risk score.

Flask Application (`app.py`)

Coordinates the analysis pipeline, manages parallel execution using `ThreadPoolExecutor`, maintains the cache, and exposes REST endpoints.

Frontend

The frontend built with React and Vite renders analysis results using animated panels powered by Framer Motion and styled with Tailwind CSS.

Chapter 5

System Implementation

ProblySUS has been developed using a modular and scalable architecture that ensures robustness and flexibility. The system maintains a clear separation of concerns, with the frontend web application and the backend Flask API hosted as independent servers communicating through RESTful endpoints. All data exchange between client and server is conducted in JSON format, enabling seamless serialisation and consistent rendering across the web interface and browser extension.

The implementation followed a structured development lifecycle: beginning with requirement analysis to identify key detection signals, followed by modular design ensuring each analysis function is independently testable and replaceable, then incremental feature development across two phases, and finally rigorous integration testing and performance optimisation.

5.1 Modular Implementation

ProblySUS was implemented as four major components: frontend web application, backend API and analysis pipeline, browser extension, and the threat intelligence data layer.

5.1.1 Frontend Development (React + Vite + Tailwind CSS v4)

The following figure shows the initial URL submission interface used to start a scan.

The frontend is built with React using Vite as the build tool and Tailwind CSS for styling. Axios handles HTTP communication with the backend.

The home page provides a minimal interface containing a single URL input bar. The input supports deep-linking where the application reads the `?url=` query parameter and automatically triggers analysis.

The following figure presents the main risk result view with score gauge and label.

Upon receiving backend results, the dashboard renders the composite risk score gauge, colour-coded risk label, recommendation string, and an ordered list of reasons.

The following figure highlights runtime behavior indicators such as redirects and external

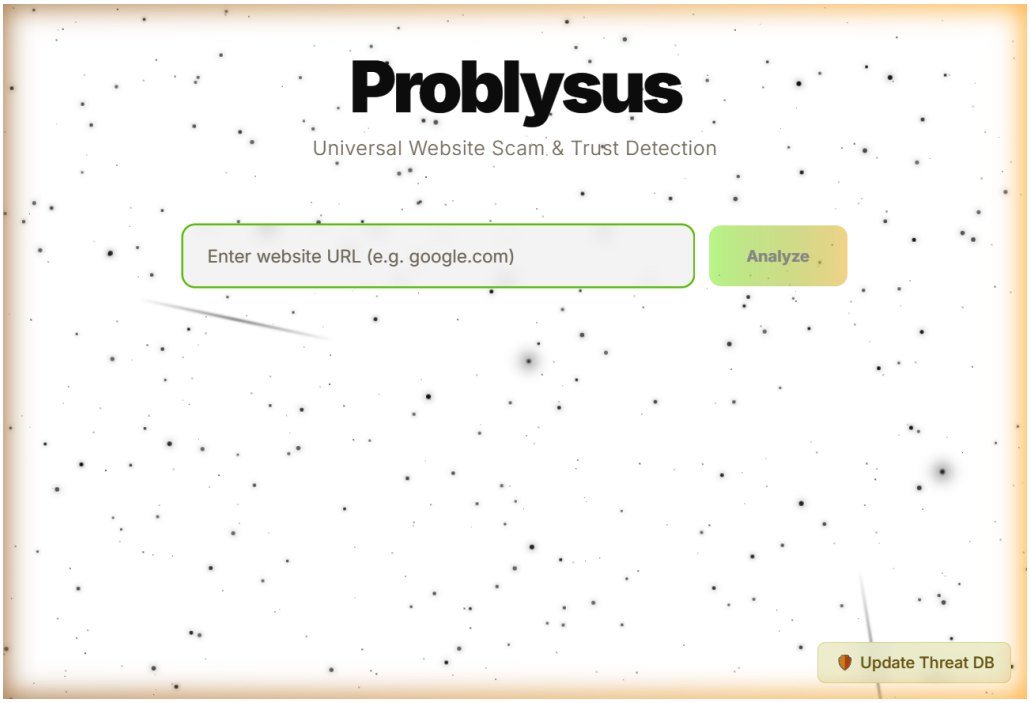

Figure 5.1: Home Page / URL Input Screen

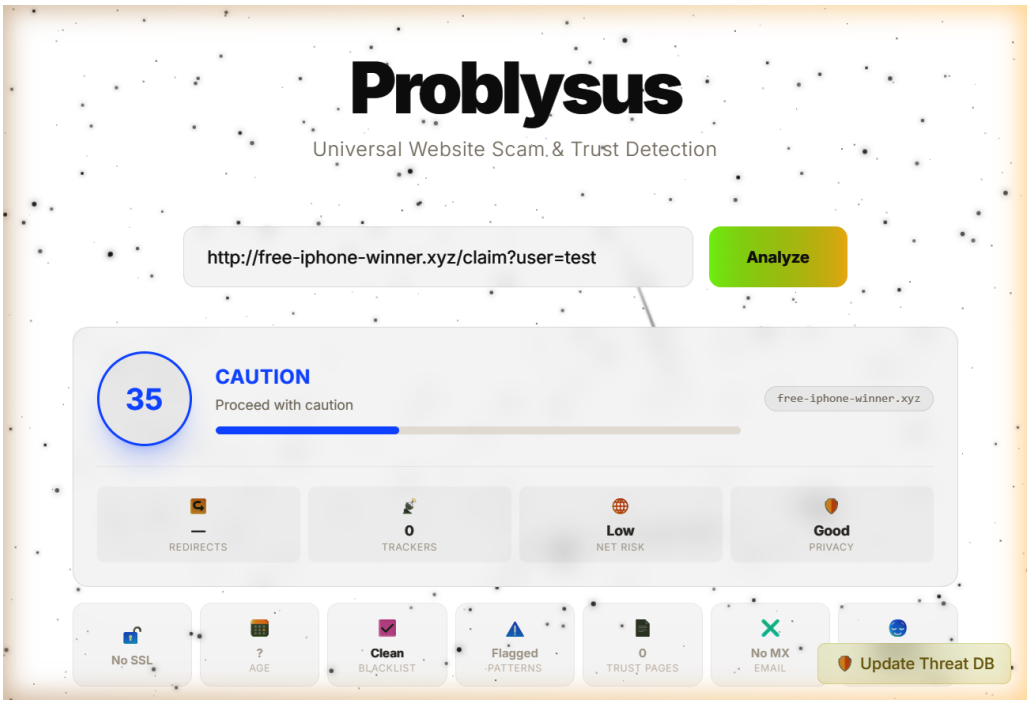

Figure 5.2: Analysis Results Dashboard — Score Gauge and Risk Label

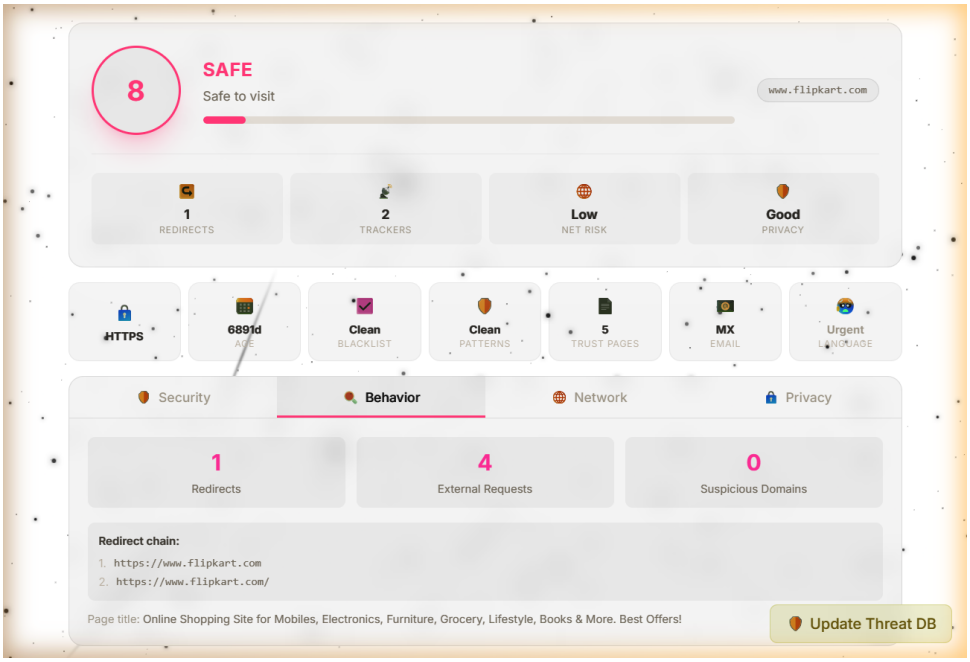

Figure 5.3: Behavior Panel — Redirects, External Requests, Suspicious Domains

requests.

The Behavior Panel displays redirect count, external requests, suspicious domains, redirect chains, and the final page title.

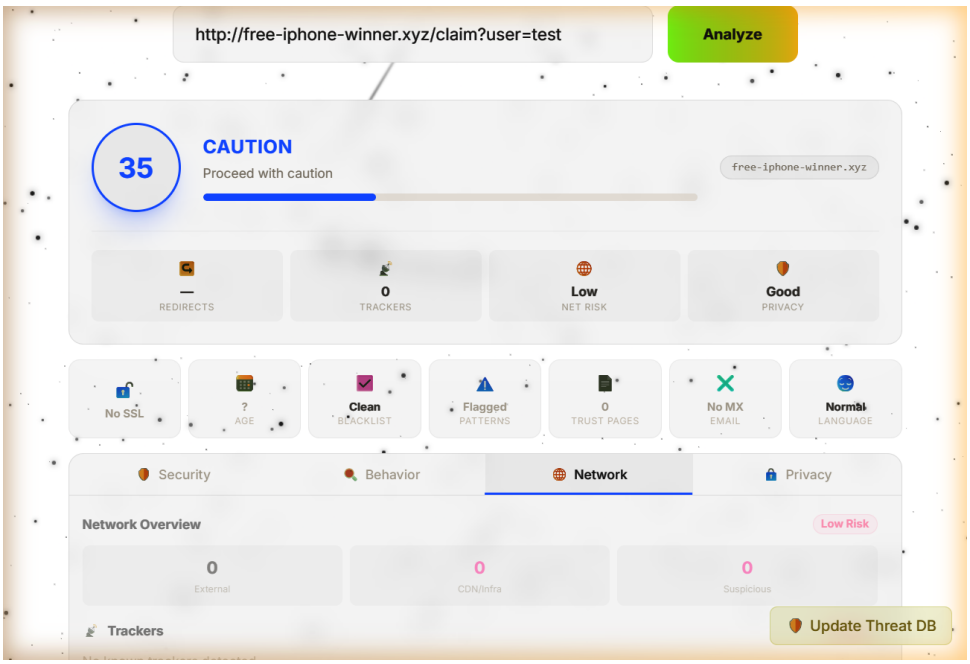

Figure 5.4: Network Panel — Domain Classification and Risk Level

The following figure shows network-level domain classification and overall network risk.

The Network Panel presents the external domain count, safe infrastructure domains, suspicious domains, and overall network risk classification.

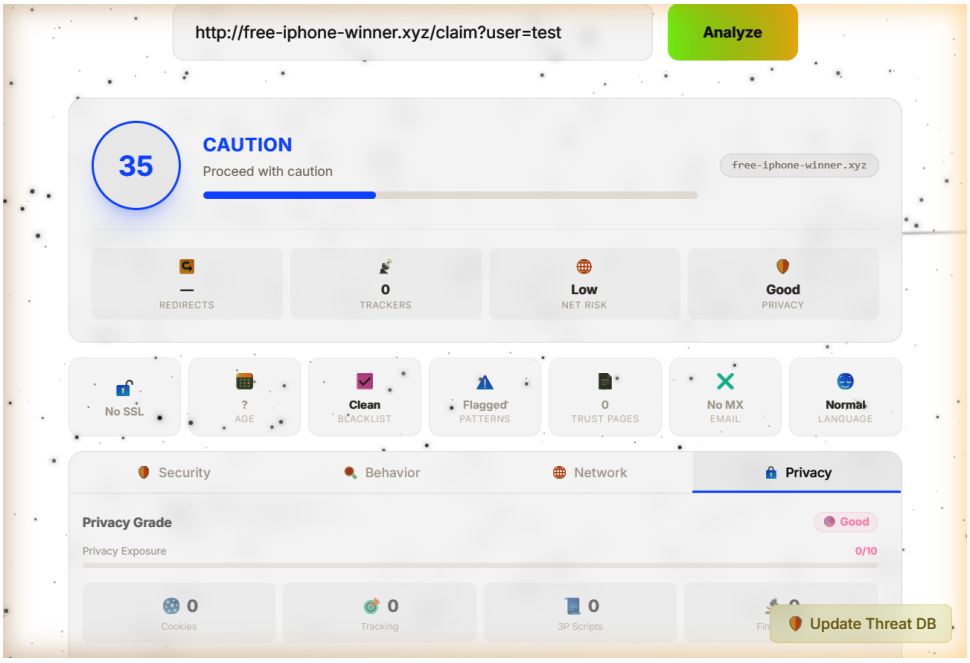

Figure 5.5: Privacy Panel — Exposure Bar, Fingerprinting Indicators, Cookie Counts

The following figure summarizes privacy exposure using cookies, scripts, and fingerprinting signals.

The Privacy Panel displays privacy grade, exposure bar, cookie counts, third-party scripts, and fingerprinting signals.

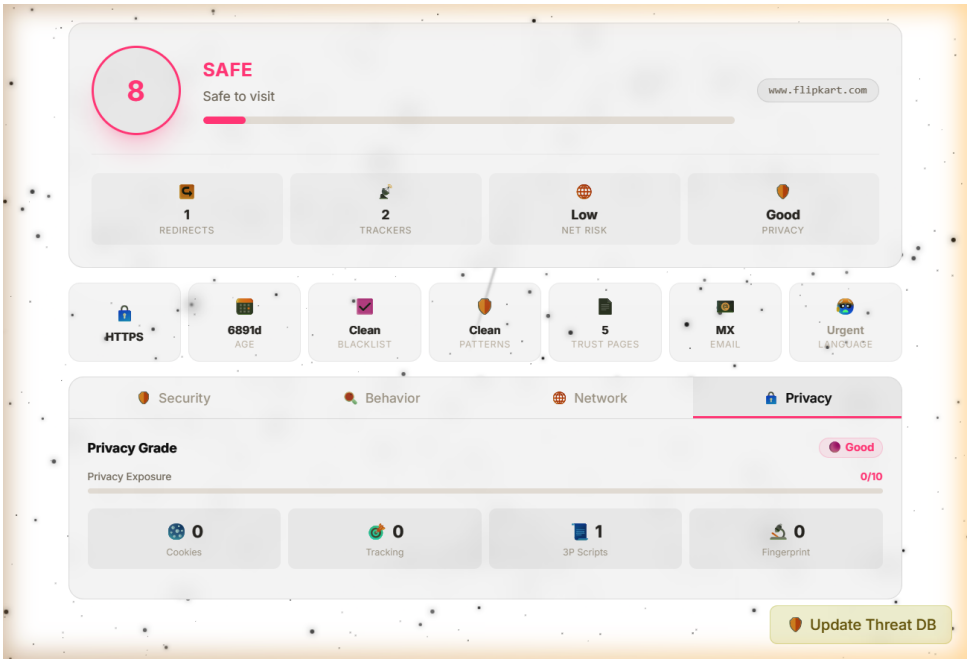

Figure 5.6: Tracker Panel — Named Tracker Detection

The following figure lists detected tracker services and their matched domains.

The Tracker Panel shows detected trackers including tracker name, matched domain, and HTML element type.

5.1.2 Backend Development (Python / Flask)

app.py — Route Structure & Parallel Execution

```python
# ProblySUS Backend — app.py  (Flask API)
from flask import Flask, request, jsonify
from flask_cors import CORS
from concurrent.futures import ThreadPoolExecutor
import logging, threading, datetime
from urllib.parse import urlparse

# Logic Modules
from logic.validator import validate_url, check_https_ssl
from logic.whois_checker import check_domain_age
from logic.pattern_checker import check_patterns
from logic.blacklist_checker import check_blacklist, force_reload_blacklist
from logic.scorer import calculate_risk_score
from logic.behavior_checker import analyze_behavior
from logic.tracker_checker import detect_trackers
from logic.network_checker import analyze_network
from logic.privacy_checker import analyze_privacy

app = Flask(__name__)
CORS(app)

CACHE_TTL_MINUTES = 30
analysis_cache = {}

@app.route("/analyze", methods=["POST"])
def analyze_url():
    data = request.get_json(silent=True)
    url = data["url"]
    fast_mode = request.args.get('fast', 'false').lower() == 'true'

    valid_url, error = validate_url(url)

    # Fast synchronous checks
    is_https, https_details = check_https_ssl(valid_url)
    patterns = check_patterns(valid_url)
    is_blacklisted = check_blacklist(hostname)

    # Slower checks executed concurrently via ThreadPoolExecutor
    with ThreadPoolExecutor(max_workers=5) as executor:
        whois_future  = executor.submit(check_domain_age, hostname)
        dns_future    = executor.submit(check_dns_records, hostname)
        html_future   = executor.submit(fetch_html)

        html_content = html_future.result(timeout=6)

        content_future  = executor.submit(check_content_trust, valid_url, html_content=html_content)
        tracker_future  = executor.submit(detect_trackers, html_content or "")
        privacy_future  = executor.submit(analyze_privacy, valid_url, html_content=html_content)

        if fast_mode:
            behavior_result = {"error": "Skipped in fast mode"}
        else:
            behavior_result = analyze_behavior(valid_url, html_content=html_content)

        age_days, _   = whois_future.result(timeout=10)
        tracker_result = tracker_future.result(timeout=10)
        privacy_result = privacy_future.result(timeout=10)

    network_result = analyze_network(behavior_result)
    score, label, reasons = calculate_risk_score(check_results)

    return jsonify(result), 200
```

Figure 5.7: app.py Route Structure and Parallel Execution

The backend is implemented in Flask. The primary endpoint `POST /analyze` orchestrates the entire analysis pipeline.

Fast checks execute first, while slower network-dependent checks are executed concurrently using `ThreadPoolExecutor`. HTML content is fetched once and reused across multiple modules.

scorer.py — Weighted Penalty Calculation

```
"""
Risk Scoring Engine — ProblySUS
================================
Weight budget (excluding blacklist override):

  Category          | Max Penalty | Rationale
  ------------------|-------------|-----------------------------------------
  HTTPS/SSL         |  10         | No encryption = real risk
  Domain age        |  20         | Very new domains are high-signal for phishing
  DNS/MX records    |   8         | Missing email infra = mild indicator
  URL patterns      |  18         | IP-based URLs, phishing keywords = strong signal
  Content analysis  |  15         | Urgency language, sensitive data requests
  Behavior          |  15         | Redirects, suspicious runtime requests
  Network           |  10         | External domain risk profile
  Trackers          |   2         | Excessive = mild privacy concern
  Privacy           |   2         | Fingerprinting, tracking cookies
  Malicious scripts |  10         | Credential stealers, obfuscated code
  ------------------|-------------|
  TOTAL (soft max)  | 100         |
"""

def calculate_risk_score(check_results):
    score = 0
    reasons = []

    # 1. BLACKLIST — Hard override. Instant Fraudulent. No appeal.
    if is_listed:
        return 100, "Fraudulent", reasons

    # 2. HTTPS / SSL CHECK  (max: 10)
    if not https_valid:
        score += 10
        reasons.append("⚠ Not using HTTPS — data sent over this site is not encrypted")

    # 3. DOMAIN AGE  (max: 20)
    if domain_age < 7:     score += 20    # Brand new — under 1 week
    elif domain_age < 30:  score += 15    # Under 1 month
    elif domain_age < 90:  score += 8     # Under 3 months

    # 5. URL PATTERNS  (max: 18)
    if patterns.get("ip_based"):        pattern_score += 10
    if patterns.get("suspicious_tld"):  pattern_score += 4
    if patterns.get("hyphens"):         pattern_score += 3
    score += min(18, pattern_score)

    # 7. BEHAVIOR ANALYSIS  (max: 15)
    if redirect_count >= 10:  behavior_score += 8
    elif redirect_count >= 7: behavior_score += 4
    if len(suspicious_domains) >= 3: behavior_score += 7
    score += min(18, behavior_score)

    # Compound signal amplification
    if domain_age < 30 and redirect_count >= 3:
        score += 5   # New domain + aggressive redirects = high phishing risk

    score = min(100, score)

    if score >= 80:   label = "Fraudulent"
    elif score >= 50: label = "Suspicious"
    elif score >= 20: label = "Caution"
    else:             label = "Safe"

    return score, label, reasons
```

Figure 5.8: scorer.py Weighted Penalty Calculation

All module outputs are passed to `calculate_risk_score()` in `scorer.py`, which applies weighted penalties and compound signal amplification to produce the final score and label.

5.1.3 Analysis Module Implementation

validator.py & pattern_checker.py — URL Validation & Lexical Inspection

```python
# validator.py — URL Normalisation & TLS Verification
from urllib.parse import urlparse
import requests

def validate_url(url):
    """Validates URL and prepends https:// if missing."""
    url = url.strip()
    if not url.startswith(("http://", "https://")):
        url = "https://" + url                    # Auto-prepend HTTPS
    result = urlparse(url)
    if all([result.scheme, result.netloc]):
        if "." not in result.netloc and result.netloc != "localhost":
            return None, "Invalid URL: Not a valid hostname"
        return url, None
    return None, "Invalid URL format"

def check_https_ssl(url):
    """Checks HTTPS and validates SSL certificate via live request."""
    if not url.startswith("https://"):
        return False, "Not using HTTPS"
    try:
        requests.get(url, timeout=7, verify=True, allow_redirects=True)
        return True, "Valid HTTPS and SSL certificate"
    except requests.exceptions.SSLError as e:
        return False, f"SSL Certificate Error: {str(e)[:120]}"
    except (requests.exceptions.ConnectionError, requests.exceptions.Timeout):
        return True, "HTTPS present but site is currently unreachable"
```

```python
# pattern_checker.py — Lexical URL Inspection for Phishing Signals
SUSPICIOUS_TLDS = {"xyz", "tk", "ml", "ga", "cf", "gq", "pw", "top", "click", "buzz"}
PHISHING_KEYWORDS = ["login", "secure", "verify", "account", "update",
                     "confirm", "banking", "paypal", "amazon", "signin",
                     "free", "winner", "claim", "prize", "urgent"]

def check_patterns(url):
    """Performs lexical URL analysis to detect suspicious patterns."""
    parsed = urlparse(url)
    hostname = parsed.hostname or ""
    results = {
        "ip_based":      bool(re.match(r"^\d{1,3}(\.\d{1,3}){3}$", hostname)),
        "suspicious_tld": hostname.rsplit(".", 1)[-1].lower() in SUSPICIOUS_TLDS,
        "hyphens":       hostname.count("-") > 3,
        "keywords":      [kw for kw in PHISHING_KEYWORDS
                          if kw in hostname.lower() or kw in url.lower()],
    }
    return results
```

Figure 5.9: validator.py and pattern_checker.py Implementation

The `validator.py` module normalises URLs and verifies HTTPS/TLS certificates before further analysis.

The `pattern_checker.py` module performs lexical URL inspection to detect suspicious patterns.

The behavior module launches a headless Chromium browser using Playwright and intercepts redirect responses and network requests.

The privacy module inspects cookies, tracking scripts, and browser fingerprinting techniques.

behavior_checker.py — Playwright Headless Browser Intercept Logic

```python
"""
Behavior Analysis Module — ProblySUS
Analyzes runtime website behavior using Playwright headless browser.
Tracks redirects, monitors network requests, detects suspicious domains.
"""
from playwright.sync_api import sync_playwright
from urllib.parse import urlparse

SUSPICIOUS_TLDS = {"tk","ml","ga","cf","xyz","top","work","click","pw","buzz"}

def analyze_behavior(url, html_content=None):
    """Launch headless Chromium, intercept all network activity."""
    redirect_chain = []
    external_domains = set()
    origin_domain = urlparse(url).hostname or ""

    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()

        # Intercept HTTP redirect responses (3xx)
        def on_response(response):
            if 300 <= response.status < 400:
                redirect_chain.append(response.url)
        page.on("response", on_response)

        # Track client-side navigations (meta refresh, JS redirects)
        def on_framenavigated(frame):
            if frame == page.main_frame:
                frame_url = frame.url
                if frame_url not in redirect_chain and frame_url != url:
                    redirect_chain.append(frame_url)
        page.on("framenavigated", on_framenavigated)

        # Track all outbound network requests
        def on_request(request):
            req_domain = urlparse(request.url).hostname or ""
            if req_domain and req_domain != origin_domain:
                clean_req = req_domain.lstrip("www.")
                if not clean_req.endswith(origin_domain.lstrip("www.")):
                    external_domains.add(req_domain)
        page.on("request", on_request)

        page.goto(url, wait_until="domcontentloaded", timeout=5000)
        page.wait_for_timeout(3000)   # Capture async/deferred requests
        browser.close()

    suspicious = [d for d in external_domains if _is_suspicious_domain(d)]
    return {
        "redirect_count":        max(0, len(redirect_chain) - 1),
        "redirect_chain":        redirect_chain[:10],
        "external_request_count": len(external_domains),
        "suspicious_domains":    suspicious,
        "page_title":            page.title(),
    }
```

Figure 5.10: behavior_checker.py Playwright Intercept Logic

5.1.4 Threat Intelligence Data Layer

The threat intelligence layer downloads the URLhaus blacklist feed and loads it into memory for fast domain lookups.

privacy_checker.py — Cookie & Browser Fingerprinting Detection

```python
"""
Privacy Analyzer Module — ProblySUS
Analyzes cookies, third-party scripts, and browser fingerprinting attempts.
"""
import re, requests
from urllib.parse import urlparse
from bs4 import BeautifulSoup

# Known fingerprinting API patterns in JavaScript
FINGERPRINT_PATTERNS = [
    (r"\.toDataURL\s*\(",   "canvas"),
    (r"getContext\s*\(\s*['\"]webgl", "webgl"),
    (r"AudioContext",        "audio"),
    (r"navigator\.plugins", "plugins"),
    (r"screen\.colorDepth", "screen"),
    (r"navigator\.hardwareConcurrency", "hardware"),
    (r"RTCPeerConnection",  "webrtc"),
]

TRACKING_COOKIE_PATTERNS = [
    "_ga","_gid","_gat","_fbp","_fbc","__utma","__utmb",
    "_hjid","_tt_","_ttp","NID","IDE","DSID",
]

def analyze_privacy(url, html_content=None):
    response = requests.get(url, timeout=8, headers=BROWSER_HEADERS)
    if html_content is None:
        html_content = response.text

    # 1. Cookie Analysis — extract Set-Cookie headers
    cookie_names = [v.split("=")[0].strip()
                    for k, v in response.headers.items()
                    if k.lower() == "set-cookie"]
    tracking = [n for n in cookie_names
                if any(p.lower() in n.lower() for p in TRACKING_COOKIE_PATTERNS)]

    # 2. Third-Party Script Analysis via BeautifulSoup
    soup = BeautifulSoup(html_content, "html.parser")
    third_party = {urlparse(s["src"]).hostname
                   for s in soup.find_all("script", src=True)
                   if not urlparse(s["src"]).hostname.endswith(origin)}

    # 3. Fingerprinting Detection — regex scan over page HTML/JS
    detected = {technique for pattern, technique in FINGERPRINT_PATTERNS
                if re.search(pattern, html_content, re.IGNORECASE)}

    # 4. Compute Privacy Grade
    grade_score  = (3 if len(tracking) >= 5 else 1 if len(tracking) >= 2 else 0)
    grade_score += (3 if len(third_party) >= 10 else 2 if len(third_party) >= 5 else 0)
    grade_score += (3 if len(detected) >= 4 else 1 if len(detected) >= 2 else 0)

    privacy_grade = ("invasive" if grade_score >= 7
                     else "poor" if grade_score >= 4
                     else "moderate" if grade_score >= 2
                     else "good")

    return {
        "cookie_count": len(cookie_names),
        "tracking_cookie_count": len(tracking),
        "third_party_script_count": len(third_party),
        "fingerprinting_signals": sorted(detected),
        "fingerprinting_score": min(5, len(detected)),
        "privacy_grade": privacy_grade,
    }
```

Figure 5.11: privacy_checker.py Cookie and Fingerprinting Detection

updater.py — URLhaus Feed Download & CSV Extraction

```python
# updater.py — Threat Intelligence Data Layer
import requests, csv, json, io, zipfile
from urllib.parse import urlparse
from datetime import datetime

URLHAUS_CSV_URL = "https://urlhaus.abuse.ch/downloads/csv/"
BLACKLIST_FILE  = "data/blacklist_sources.json"

def update_blacklist_source():
    """
    Fetches the latest URLhaus blacklist, extracts CSV from ZIP,
    parses domain entries and persists them as JSON.
    Custom/manually-added entries are never overwritten.
    """
    response = requests.get(URLHAUS_CSV_URL, timeout=60)
    response.raise_for_status()

    # URLhaus /csv/ endpoint returns a ZIP archive
    is_zip = response.content[:2] == b"PK"    # ZIP magic bytes
    if is_zip:
        with zipfile.ZipFile(io.BytesIO(response.content)) as zf:
            csv_text = zf.read(zf.namelist()[0]).decode("utf-8", errors="replace")
    else:
        csv_text = response.text

    reader = csv.reader(io.StringIO(csv_text, newline=""))
    new_domains = {}

    for row in reader:
        if not row or row[0].startswith("#") or len(row) < 6:
            continue
        # Columns: id, dateadded, url, url_status, last_online, threat, ...
        url, status, threat = row[2], row[3], row[5]

        if status in ("online", "offline"):
            hostname = urlparse(url).hostname
            if hostname:
                new_domains[hostname.lower()] = {
                    "category":   f"Active Malware ({threat})" if status=="online"
                                  else f"Previously Confirmed Malicious ({threat})",
                    "source":     "URLHaus",
                    "risk_level": "Critical" if status=="online" else "High",
                    "updated_at": datetime.now().isoformat(),
                }

    # Load existing file to preserve custom_domains (never overwritten)
    existing = json.load(open(BLACKLIST_FILE)) if os.path.exists(BLACKLIST_FILE) else {}
    existing["domains"] = new_domains            # REPLACE strategy
    existing["meta"]    = {"last_updated": datetime.now().isoformat(),
                           "domain_count": len(new_domains)}

    with open(BLACKLIST_FILE, "w") as f:
        json.dump(existing, f, indent=2)

    return {"success": True, "added_count": len(new_domains)}


# Auto-update: runs 5 seconds after startup, repeats every 24 hours
def _auto_update_blacklist():
    result = update_blacklist_source()
    timer = threading.Timer(86400, _auto_update_blacklist)   # 24h reschedule
    timer.daemon = True
    timer.start()

threading.Timer(5, _auto_update_blacklist).start()
```

Figure 5.12: URLhaus Feed Download and CSV Extraction

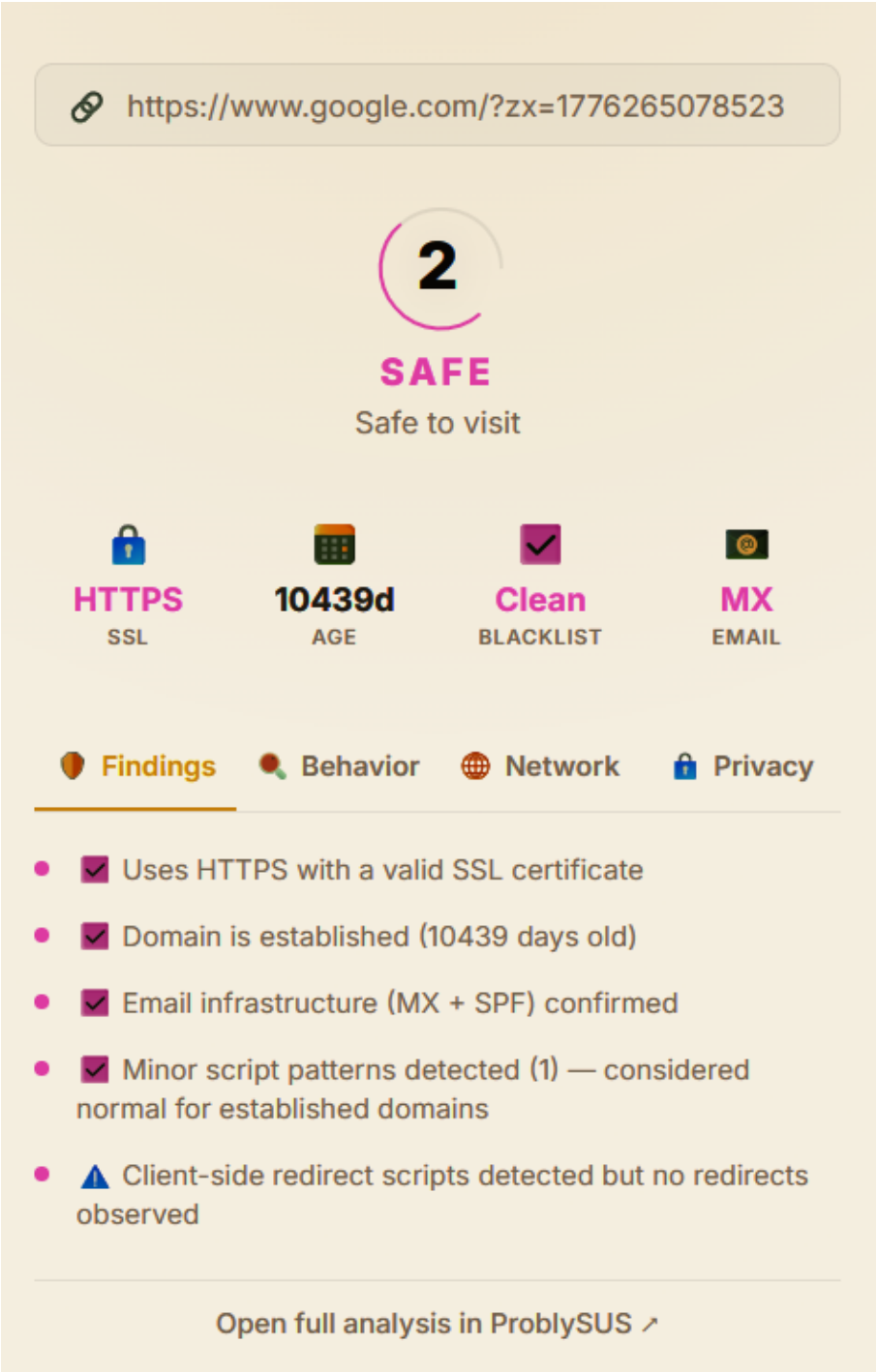

Figure 5.13: Browser Extension Popup — Overview Tab

5.1.5 Browser Extension Implementation

The following figure shows the extension overview tab with the primary risk summary.

The Chrome extension uses Manifest V3 and communicates with the Flask backend to analyze the active tab URL.

The following figure presents detailed behavior and privacy insights inside the extension

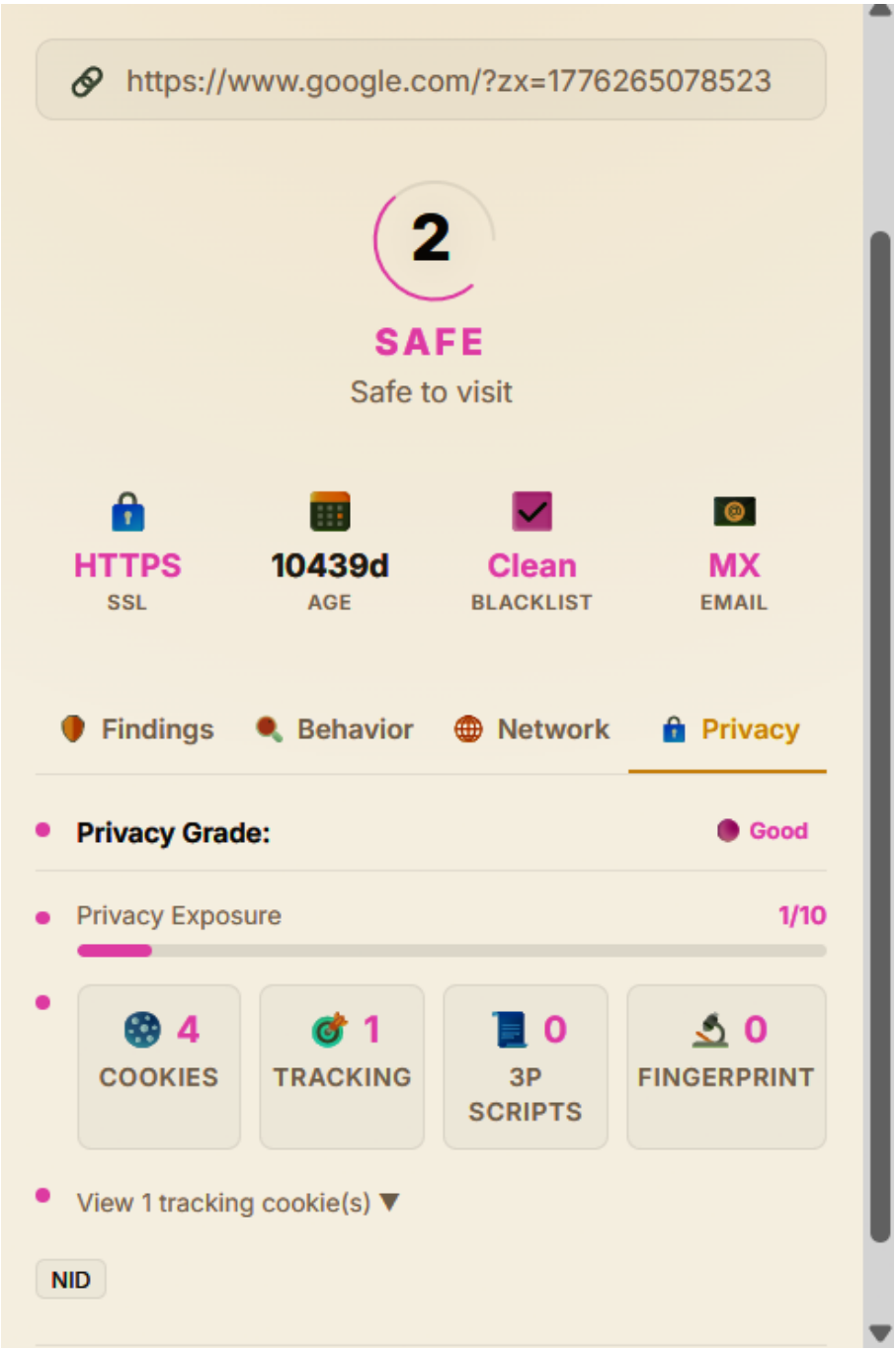

Figure 5.14: Browser Extension Popup — Behavior and Privacy Tabs

popup.

The popup UI displays the score arc, risk label, status indicators, and detailed analysis tabs.

5.1.6 Fast Mode and Caching

The following figure illustrates how fast mode and caching improve repeated scan performance.

Fast Mode bypasses Playwright runtime analysis to reduce total execution time. A result

app.py — Fast Mode & Result Cache Logic

```python
# app.py — In-Memory Result Cache & Fast Mode
import datetime

# ── Result Cache ──────────────────────────────────────────────────────
# Simple dict: { url → { "result": data, "expiry": datetime } }
analysis_cache   = {}
CACHE_TTL_MINUTES = 30          # Cached results expire after 30 minutes

# ── Cache Lookup ──────────────────────────────────────────────────────
force_scan = request.args.get('force', 'false').lower() == 'true'

if not force_scan:
    cached = analysis_cache.get(valid_url)
    if cached:
        expiry = cached.get("expiry")
        if expiry and datetime.datetime.now() < expiry:
            result = cached["result"].copy()
            result["cached"] = True              # Mark as served from cache
            return jsonify(result), 200
        else:
            analysis_cache.pop(valid_url, None)  # Expired — evict entry

# ── Fast Mode ─────────────────────────────────────────────────────────
# ?fast=true skips Playwright runtime analysis (behavior_checker).
# Reduces total latency by ~5-8 seconds for quick lookups.
fast_mode = request.args.get('fast', 'false').lower() == 'true'

if fast_mode:
    behavior_result = {
        "redirect_count":        0,
        "redirect_chain":        [],
        "external_request_count": 0,
        "suspicious_domains":    [],
        "page_title":            None,
        "error":                 "Skipped in fast mode"
    }
else:
    # Full Playwright headless browser analysis
    behavior_result = analyze_behavior(valid_url, html_content=html_content)

# ── Store Result in Cache ─────────────────────────────────────────────
analysis_cache[valid_url] = {
    "result": result,
    "expiry": datetime.datetime.now() + datetime.timedelta(minutes=CACHE_TTL_MINUTES)
}
```

Figure 5.15: Fast Mode and Result Cache Logic in app.py

cache stores recently analyzed URLs for 30 minutes to prevent redundant scans.

Chapter 6

Technologies, Stacks and Tools Used

This chapter outlines the technologies, development stacks, and tools used in the implementation of ProblySUS, spanning the backend analysis engine, React web frontend, and Chrome browser extension.

6.1 Development Environment

The development of ProblySUS was carried out primarily using Visual Studio Code as the IDE across all components — backend, frontend, and extension — chosen for its rich Python and JavaScript ecosystem, integrated terminal, and Git support. The backend Python environment was managed using `venv` to isolate dependencies. Node.js and npm were used for frontend dependency management and running the Vite development server. Git was used for version control throughout the project.

6.2 Frontend Technologies

6.2.1 React 19 + Vite 7

The web frontend is built with React 19, a component-based JavaScript library that enables reactive, state-driven UIs. Vite 7 serves as the build tool and development server, providing near-instant hot module replacement and optimised production builds.

The following figure shows the web application's URL input page where analysis begins.

6.2.2 Tailwind CSS v4

Tailwind CSS v4 is used for styling through utility classes. Its just-in-time compilation ensures minimal CSS bundle size and efficient rendering.

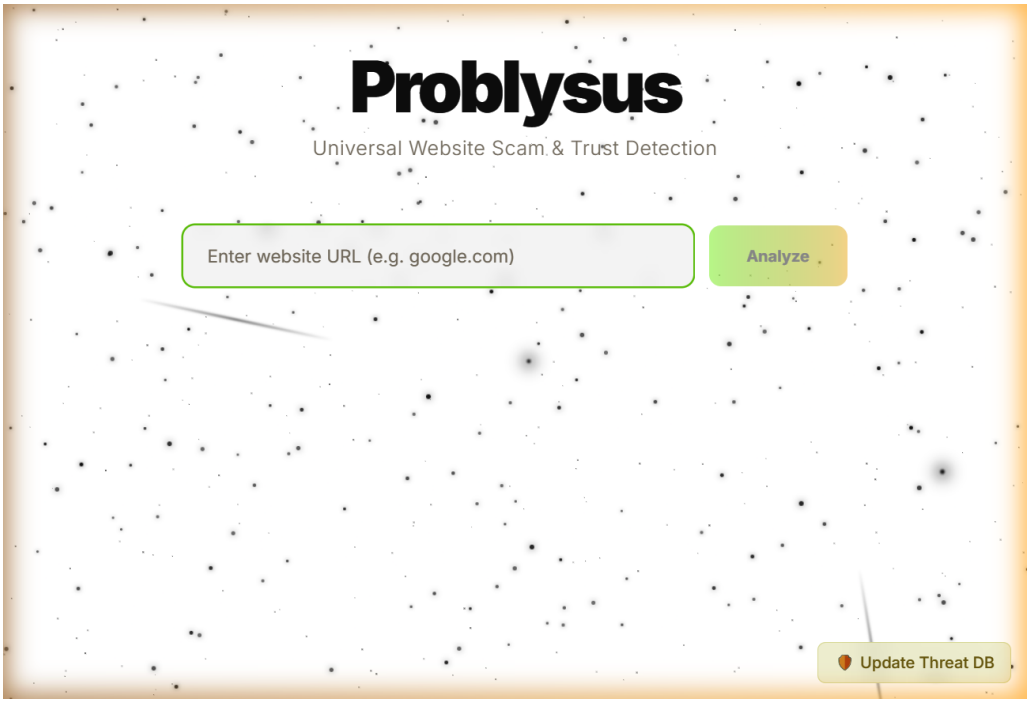

Figure 6.1: Web Application Home Page — URL Input Interface

6.2.3 Framer Motion

Framer Motion provides UI animations such as fade-in panel transitions and animated privacy exposure bars.

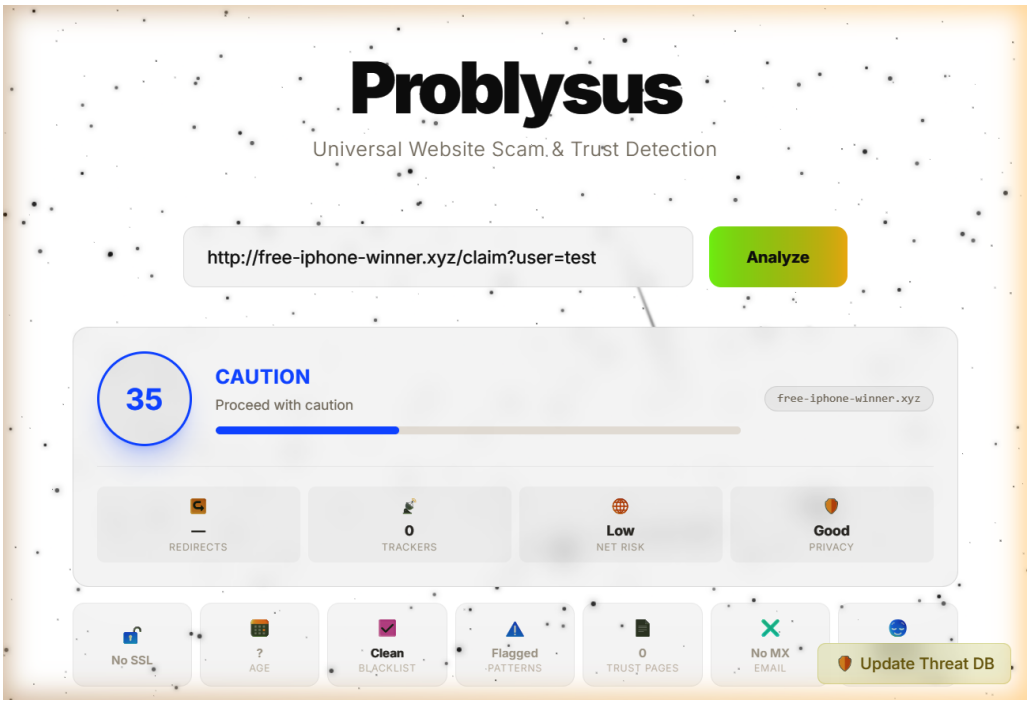

Figure 6.2: Analysis Result Dashboard with Animated Panels

The following figure demonstrates the dashboard layout used to present scan outputs.

6.2.4 Axios

Axios handles all HTTP communication between the React frontend and the Flask backend by sending POST requests to the analysis endpoint and receiving structured JSON responses.

6.3 Backend Technologies

6.3.1 Python 3 / Flask

The backend API is developed using Python 3 and the Flask micro-framework. Flask-CORS allows cross-origin requests from the React frontend and browser extension.

6.3.2 Concurrent Execution — ThreadPoolExecutor

Python's `ThreadPoolExecutor` enables concurrent execution of network-dependent modules such as WHOIS lookup, DNS queries, and content analysis, reducing total processing time.

6.3.3 Playwright + Chromium

Playwright is used to launch a headless Chromium browser to simulate real browsing behavior and capture redirects and external requests.

6.3.4 Analysis Libraries

Library	Purpose
requests	HTTP requests and feed downloads
python-whois	Domain registration queries
dnspython	DNS lookups for MX and SPF records
beautifulsoup4	HTML parsing for analysis modules
tldextract	Domain extraction from URLs
textstat	Content readability scoring
re	Regular expression matching
csv / zipfile	URLhaus feed extraction

Table 6.1: Backend Analysis Libraries

app.py — Route Structure & Parallel Execution

```python
# ProblySUS Backend — app.py  (Flask API)
from flask import Flask, request, jsonify
from flask_cors import CORS
from concurrent.futures import ThreadPoolExecutor
import logging, threading, datetime
from urllib.parse import urlparse

# Logic Modules
from logic.validator import validate_url, check_https_ssl
from logic.whois_checker import check_domain_age
from logic.pattern_checker import check_patterns
from logic.blacklist_checker import check_blacklist, force_reload_blacklist
from logic.scorer import calculate_risk_score
from logic.behavior_checker import analyze_behavior
from logic.tracker_checker import detect_trackers
from logic.network_checker import analyze_network
from logic.privacy_checker import analyze_privacy

app = Flask(__name__)
CORS(app)

CACHE_TTL_MINUTES = 30
analysis_cache = {}

@app.route("/analyze", methods=["POST"])
def analyze_url():
    data = request.get_json(silent=True)
    url = data["url"]
    fast_mode = request.args.get('fast', 'false').lower() == 'true'

    valid_url, error = validate_url(url)

    # Fast synchronous checks
    is_https, https_details = check_https_ssl(valid_url)
    patterns = check_patterns(valid_url)
    is_blacklisted = check_blacklist(hostname)

    # Slower checks executed concurrently via ThreadPoolExecutor
    with ThreadPoolExecutor(max_workers=5) as executor:
        whois_future  = executor.submit(check_domain_age, hostname)
        dns_future    = executor.submit(check_dns_records, hostname)
        html_future   = executor.submit(fetch_html)

        html_content = html_future.result(timeout=6)

        content_future  = executor.submit(check_content_trust, valid_url, html_content=html_content)
        tracker_future  = executor.submit(detect_trackers, html_content or "")
        privacy_future  = executor.submit(analyze_privacy, valid_url, html_content=html_content)

        if fast_mode:
            behavior_result = {"error": "Skipped in fast mode"}
        else:
            behavior_result = analyze_behavior(valid_url, html_content=html_content)

        age_days, _   = whois_future.result(timeout=10)
        tracker_result = tracker_future.result(timeout=10)
        privacy_result = privacy_future.result(timeout=10)

    network_result = analyze_network(behavior_result)
    score, label, reasons = calculate_risk_score(check_results)

    return jsonify(result), 200
```

Figure 6.3: Flask app.py Route Definitions and ThreadPoolExecutor Block

behavior_checker.py — Playwright Headless Browser Intercept Logic

```python
"""
Behavior Analysis Module — ProblySUS
Analyzes runtime website behavior using Playwright headless browser.
Tracks redirects, monitors network requests, detects suspicious domains.
"""
from playwright.sync_api import sync_playwright
from urllib.parse import urlparse

SUSPICIOUS_TLDS = {"tk","ml","ga","cf","xyz","top","work","click","pw","buzz"}

def analyze_behavior(url, html_content=None):
    """Launch headless Chromium, intercept all network activity."""
    redirect_chain = []
    external_domains = set()
    origin_domain = urlparse(url).hostname or ""

    with sync_playwright() as p:
        browser = p.chromium.launch(headless=True)
        page = browser.new_page()

        # Intercept HTTP redirect responses (3xx)
        def on_response(response):
            if 300 <= response.status < 400:
                redirect_chain.append(response.url)
        page.on("response", on_response)

        # Track client-side navigations (meta refresh, JS redirects)
        def on_framenavigated(frame):
            if frame == page.main_frame:
                frame_url = frame.url
                if frame_url not in redirect_chain and frame_url != url:
                    redirect_chain.append(frame_url)
        page.on("framenavigated", on_framenavigated)

        # Track all outbound network requests
        def on_request(request):
            req_domain = urlparse(request.url).hostname or ""
            if req_domain and req_domain != origin_domain:
                clean_req = req_domain.lstrip("www.")
                if not clean_req.endswith(origin_domain.lstrip("www.")):
                    external_domains.add(req_domain)
        page.on("request", on_request)

        page.goto(url, wait_until="domcontentloaded", timeout=5000)
        page.wait_for_timeout(3000)   # Capture async/deferred requests
        browser.close()

    suspicious = [d for d in external_domains if _is_suspicious_domain(d)]
    return {
        "redirect_count":        max(0, len(redirect_chain) - 1),
        "redirect_chain":        redirect_chain[:10],
        "external_request_count": len(external_domains),
        "suspicious_domains":    suspicious,
        "page_title":            page.title(),
    }
```

Figure 6.4: Playwright Launch and Network Interception Logic

6.4 Browser Extension Technologies

The browser extension is built using Manifest V3 and plain HTML, CSS, and JavaScript for the popup interface.

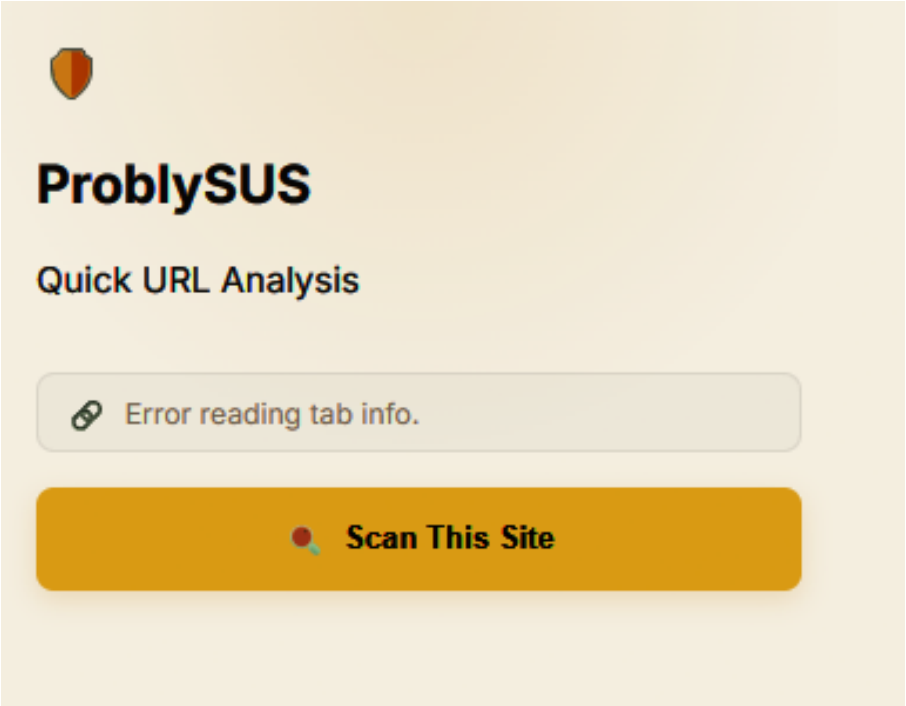

Figure 6.5: Chrome Extension Popup — Overview Tab

The following figure shows the extension overview tab with compact risk information.

The score arc is implemented using CSS transforms and transitions.

The following figure shows behavior and privacy tabs for deeper inspection inside the extension.

6.5 Threat Intelligence Tools

6.5.1 URLhaus Feed

URLhaus provides a community-maintained dataset of malicious URLs distributed as a ZIP-compressed CSV feed.

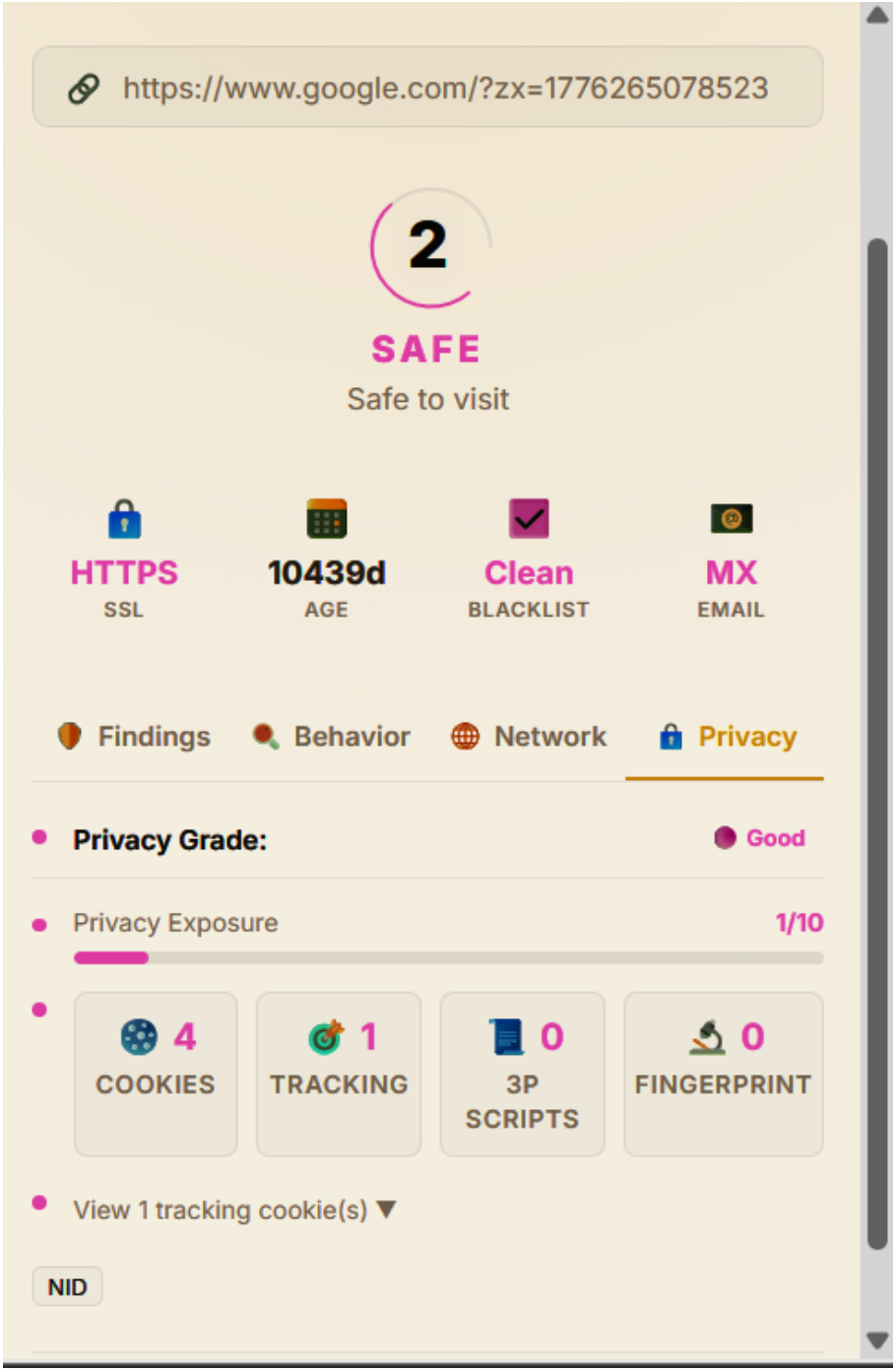

Figure 6.6: Chrome Extension Popup — Behavior and Privacy Tabs

6.5.2 Local Tracker Database

A local `trackers.json` file maps known tracker domains to human-readable tracker names used during tracker detection.

updater.py — URLhaus Feed Download & CSV Extraction

```python
# updater.py — Threat Intelligence Data Layer
import requests, csv, json, io, zipfile
from urllib.parse import urlparse
from datetime import datetime

URLHAUS_CSV_URL = "https://urlhaus.abuse.ch/downloads/csv/"
BLACKLIST_FILE  = "data/blacklist_sources.json"

def update_blacklist_source():
    """
    Fetches the latest URLhaus blacklist, extracts CSV from ZIP,
    parses domain entries and persists them as JSON.
    Custom/manually-added entries are never overwritten.
    """
    response = requests.get(URLHAUS_CSV_URL, timeout=60)
    response.raise_for_status()

    # URLhaus /csv/ endpoint returns a ZIP archive
    is_zip = response.content[:2] == b"PK"    # ZIP magic bytes
    if is_zip:
        with zipfile.ZipFile(io.BytesIO(response.content)) as zf:
            csv_text = zf.read(zf.namelist()[0]).decode("utf-8", errors="replace")
    else:
        csv_text = response.text

    reader = csv.reader(io.StringIO(csv_text, newline=""))
    new_domains = {}

    for row in reader:
        if not row or row[0].startswith("#") or len(row) < 6:
            continue
        # Columns: id, dateadded, url, url_status, last_online, threat, ...
        url, status, threat = row[2], row[3], row[5]

        if status in ("online", "offline"):
            hostname = urlparse(url).hostname
            if hostname:
                new_domains[hostname.lower()] = {
                    "category":   f"Active Malware ({threat})" if status=="online"
                                  else f"Previously Confirmed Malicious ({threat})",
                    "source":     "URLHaus",
                    "risk_level": "Critical" if status=="online" else "High",
                    "updated_at": datetime.now().isoformat(),
                }

    # Load existing file to preserve custom_domains (never overwritten)
    existing = json.load(open(BLACKLIST_FILE)) if os.path.exists(BLACKLIST_FILE) else {}
    existing["domains"] = new_domains            # REPLACE strategy
    existing["meta"]    = {"last_updated": datetime.now().isoformat(),
                           "domain_count": len(new_domains)}

    with open(BLACKLIST_FILE, "w") as f:
        json.dump(existing, f, indent=2)

    return {"success": True, "added_count": len(new_domains)}


# Auto-update: runs 5 seconds after startup, repeats every 24 hours
def _auto_update_blacklist():
    result = update_blacklist_source()
    timer = threading.Timer(86400, _auto_update_blacklist)   # 24h reschedule
    timer.daemon = True
    timer.start()

threading.Timer(5, _auto_update_blacklist).start()
```

Figure 6.7: URLhaus Feed Download and Auto-Refresh Logic

6.6 Testing Tools

Postman was used for API endpoint testing, while Chrome DevTools was used for debugging frontend components and browser extension behavior. A custom `test_scorer.py` script was used to unit-test the scoring engine independently.

6.7 Version Control and Project Management

Git was used for version control with a GitHub repository serving as the central codebase. The repository contains the full source tree including `backend/`, `frontend/`, `extension/`, and documentation files.

6.8 Deployment

ProblySUS currently runs locally with the Flask backend hosted at `http://127.0.0.1:5000` and the Vite frontend at `http://localhost:5173`. The Chrome extension is loaded as an unpacked extension via developer mode.

6.9 Security Considerations

Several security safeguards are implemented since the system analyzes potentially malicious URLs.

Playwright runs in a sandboxed environment and does not persist cookies between scans. All user input is validated before analysis, and error responses always return JSON to avoid exposing internal server details.

app.py — Global Error Handling & Input Validation Logic

```python
# ---------------------------------------------------------------------
# Global error handler — always return JSON, never HTML stack traces
# ---------------------------------------------------------------------
@app.errorhandler(Exception)
def handle_unexpected_error(e):
    logger.error(f"Unhandled exception: {e}", exc_info=True)
    return jsonify({"error": "An unexpected server error occurred. Please try again."}), 500

@app.errorhandler(404)
def handle_404(e):
    return jsonify({"error": "Endpoint not found"}), 404

@app.errorhandler(405)
def handle_405(e):
    return jsonify({"error": "Method not allowed"}), 405

# ---------------------------------------------------------------------
# Routes
# ---------------------------------------------------------------------
@app.route("/analyze", methods=["POST"])
def analyze_url():
    """
    Analyze a given URL for scam risk.
    """
    data = request.get_json(silent=True)
    if not data or "url" not in data:
        return jsonify({"error": "Request body must be JSON with a 'url' field"}), 400

    url = data["url"]

    # --- Input guards ---
    if not isinstance(url, str):
        return jsonify({"error": "URL must be a string"}), 400

    if len(url) > MAX_URL_LENGTH:
        return jsonify({"error": f"URL exceeds maximum length of {MAX_URL_LENGTH} characters"}), 400

    # 1. Validation
    valid_url, error = validate_url(url)
    if not valid_url:
        return jsonify({"error": error}), 400
```

Figure 6.8: Global Error Handling and Input Validation Logic in app.py

Chapter 7

Testing

Testing was conducted to validate the functionality, accuracy, and performance of ProblySUS. A layered testing methodology was adopted — beginning with individual module unit testing, followed by integration testing of the full analysis pipeline, and concluding with performance and usability evaluations under real-world URL scanning scenarios.

7.1 Unit Testing

Unit testing was carried out to confirm that each analysis module operated correctly in isolation before being integrated into the full pipeline. The primary objective was to verify that individual components produced accurate, expected outputs across a range of valid and edge-case inputs.

A custom `test_scorer.py` script was written to unit-test the scoring engine with synthetic `check_results` dictionaries — verifying that each signal category produced the correct penalty, compound signals triggered their amplification bonus, and blacklist hits returned the hard override of score 100 / Fraudulent regardless of other signals.

Individual modules such as `pattern_checker.py` and `validator.py` were tested with crafted URLs covering IP-based hosts, abused TLDs, excessive hyphens, missing schemes, and invalid syntax.

7.2 Integration Testing

Following unit testing, integration testing assessed the full analysis pipeline end-to-end — verifying that all modules interacted correctly, results were properly passed between components, and the final JSON response was correctly consumed by both the web frontend and the browser extension.

Scenario 1 — Known Malicious URL (Blacklist Hit): A domain present in the URLhaus feed was submitted. The blacklist checker correctly identified it, the scorer applied the hard override, and the frontend rendered a score of 100 with the *Fraudulent* label.

Scenario 2 — Newly Registered Phishing-Pattern Domain: A URL using an abused TLD (.xyz), multiple hyphens, and registered within 7 days was submitted. The pipeline ac-

Module	Test Case Description	Result
validator.py	Accepts valid URLs, rejects malformed inputs, normalises missing schemes	Pass
pattern_checker.py	Detects IP hosts, abused TLDs, hyphens, and phishing keywords correctly	Pass
whois_checker.py	Returns correct domain age in days; handles WHOIS unavailable gracefully	Pass
dns_checker.py	Correctly identifies presence/absence of MX and SPF records	Pass
blacklist_checker.py	Returns match for known malicious domain; returns clean for safe domain	Pass
content_checker.py	Detects urgency phrases, trust pages, and sensitive form fields correctly	Pass
behavior_checker.py	Captures redirect chain and external domains; falls back gracefully without Playwright	Pass
privacy_checker.py	Identifies tracking cookies, fingerprinting API patterns, and third-party scripts	Pass
tracker_checker.py	Matches known tracker domains against local database and returns tracker name	Pass
scorer.py	Produces correct scores, labels, compound penalties, and blacklist override	Pass

Table 7.1: Unit Testing Results for ProblySUS Modules

cumulated penalties across domain age, URL patterns, and DNS configuration, producing a Suspicious/Fraudulent verdict.

Scenario 3 — Legitimate Established Website: A well-known website was submitted. The system confirmed HTTPS validity, established domain age, and clean network behavior, producing a Safe score.

Scenario 4 — JavaScript Redirect Site: A site performing JavaScript-based redirects was submitted. The Playwright behavior module successfully detected redirect chains and captured external domains.

Scenario 5 — Fast Mode vs Full Mode: The same URL was tested with and without `?fast=true`. Fast mode produced a result in approximately 4 seconds while full mode produced a more detailed result in around 10 seconds.

All integration tests yielded stable results across repeated runs.

7.3 Performance Testing

Performance testing evaluated system responsiveness, concurrency handling, and reliability under realistic scanning conditions.

Test Scenario	Result
Fast Mode scan (Playwright skipped)	Completed in 3–5 seconds
Full scan (Playwright included)	Completed in 8–12 seconds
Cached URL re-scan	Returned instantly (<50 ms)
Simultaneous requests	Handled concurrently without crashes
URLhaus blacklist auto-refresh	Completed in background without blocking requests
Behavior module timeout	Graceful fallback after 6 seconds
Average API response time (fast mode)	350–700 ms excluding Playwright

Table 7.2: Performance Testing Results

These results confirm the system's readiness for real-world usage.

7.4 Usability Testing

Usability testing was performed with a group of students and general internet users to assess the ease of use and interpretability of results.

- Approximately 90% of testers found the risk label and color-coded score immediately understandable.
- Around 85% considered the reasons list helpful in understanding why a site received its score.
- 80% successfully used the browser extension to scan a site directly from their browser tab on the first attempt.
- 100% of participants expressed interest in using the tool regularly when encountering unfamiliar URLs.
- Testers identified the Privacy Panel and Tracker Panel as particularly informative features.

7.5 Bug Fixes and Improvements

During testing, several bugs and performance issues were identified and resolved.

Issue Identified	Action Taken
URLhaus feed ZIP caused CSV reader failure	Added zipfile extraction before parsing
Behavior module timeout on blocked sites	Added 6-second hard timeout and fallback
False positives on obfuscated JS	Made penalty context-aware for older domains
Redirect penalty too aggressive	Increased redirect threshold to 7+
Extension popup crash when backend offline	Added try/catch and user-friendly error message
Multiple HTML fetches by modules	Refactored to fetch HTML once in app.py
Privacy exposure bar not animating	Reset animation state before rendering

Table 7.3: Bug Fixes and Enhancements

These improvements significantly enhanced system reliability and user experience across multiple testing iterations.

Future Scope

While the current version of ProblySUS establishes a strong multi-signal foundation for URL scam detection, several strategic enhancements are planned to extend its reach and capability.

- **Machine Learning-Based Scoring:** The current rule-based scoring engine could be augmented or replaced with a trained machine learning classifier such as Random Forest or XGBoost. Training the model on a labelled dataset of phishing and legitimate URLs could improve detection accuracy beyond fixed threshold penalties.
- **Public Cloud Deployment:** Deploying the backend to a cloud platform such as AWS, Google Cloud Platform, or Render would make ProblySUS publicly accessible without requiring users to run a local server. This would significantly widen its user base and allow the browser extension to be used by anyone.
- **Real-Time Passive Protection via Extension:** The browser extension currently requires users to manually click the "Scan" button. A future version could monitor tab navigation events and proactively warn users before they land on a risky page, providing real-time safe-browsing protection.
- **Expanded Threat Intelligence Feeds:** Integrating additional threat intelligence sources such as PhishTank, OpenPhish, or Emerging Threats would improve blacklist coverage, particularly for newly launched phishing campaigns that may not yet appear in the URLhaus dataset.
- **Historical Domain Reputation Analysis:** Cross-referencing domains against historical reputation databases such as the Wayback Machine and historical hosting records could provide additional trust signals beyond the current WHOIS-based domain age analysis.
- **API Rate Limiting and Authentication:** For public deployment, implementing API key authentication and per-key rate limiting would prevent abuse of the service and ensure fair resource allocation among users.
- **User Reporting and Feedback Loop:** Introducing a user feedback system would allow users to report false positives and false negatives. This feedback could be used to continuously refine scoring thresholds and improve detection accuracy.
- **Multilingual URL and Content Analysis:** Extending the content analysis module to detect phishing indicators across multiple languages would enhance detection accuracy for non-English phishing campaigns, which are increasingly common.

Robust privacy controls will also be prioritized as the system evolves. These include enforcing HTTPS for all public API endpoints, encrypting stored result logs, and ensuring compliance with modern data protection standards when transitioning from local deployment to a cloud-based infrastructure.

Conclusion

ProblySUS stands as a demonstration of how a well-designed, multi-signal approach can meaningfully improve upon the fragmented and insufficient URL safety tools available to everyday internet users today. By combining domain intelligence, DNS analysis, URL pattern recognition, live content inspection, threat feed lookups, headless browser simulation, network profiling, privacy auditing, and tracker identification into a single unified pipeline, the system delivers a comprehensive and explainable risk assessment that no single-signal tool can match.

A key focus throughout development was building a strictly modular architecture, with each analysis concern isolated in its own independent module. This design ensures ease of testing, maintenance, and future extension without introducing cascading side effects. The frontend was developed using React 19 and Tailwind CSS v4, focusing on clarity and immediate interpretability of results for non-technical users. The backend, implemented using Python and Flask, leverages concurrent execution and intelligent caching to deliver analysis results within a practical time frame. The browser extension further enhances usability by integrating the scanning capability directly into the user's browsing context, allowing one-click analysis of the active tab without leaving the browser. Comprehensive testing, including unit, integration, performance, and usability evaluations, validated the reliability, accuracy, and resilience of all system components.

In summary, ProblySUS empowers users with transparent and actionable insights about the websites they visit, bridging the gap between passive blacklist-based tools and proactive, multi-dimensional threat analysis. The system establishes a strong foundation for a smarter and more accessible approach to online security in an era where scam and phishing attacks continue to grow in both volume and sophistication.

Bibliography

[1] I. Corona, G. Giacinto, and F. Roli, "Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues," *Information Sciences*, vol. 239, pp. 201–225, 2013.

[2] M. A. Rajab, L. Ballard, N. Jagpal, P. Mavrommatis, D. Nojiri, N. Provos, and L. Schmidt, "Trends in circumventing web-malware detection," Google Technical Report, 2011.

[3] M. Lison and J. Tiedemann, "OpenNMT: Open-source toolkit for neural machine translation," in *Proc. ACL 2016 System Demonstrations*, Berlin, Germany, 2016.

[4] N. Chou, R. Ledesma, Y. Teraguchi, and J. C. Mitchell, "Client-side defense against web-based identity theft," in *Proc. 11th Annual Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2004.

[5] F. Roesner, T. Kohno, and D. Wetherall, "Detecting and defending against third-party tracking on the web," in *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, San Jose, CA, USA, 2012, pp. 155–168.

[6] A. Oest, Y. Safaei, A. Doupe, G. J. Ahn, B. Wardman, and G. Warner, "Inside a phisher's mind: Understanding the anti-phishing ecosystem through phishing kit analysis," in *Proc. APWG Symposium on Electronic Crime Research (eCrime)*, 2018.

[7] S. Dhalwani and S. Khan, "A survey on URL-based phishing detection techniques using machine learning," *International Journal of Computer Applications*, vol. 975, p. 8887, 2020.

[8] Abuse.ch, "URLhaus — A project with the goal of sharing malicious URLs," [Online]. Available: `https://urlhaus.abuse.ch`. [Accessed: March 2026].

[9] Microsoft Playwright Team, "Playwright: Fast and reliable end-to-end testing for modern web apps," [Online]. Available: `https://playwright.dev`. [Accessed: March 2026].

[10] G. Acar, C. Eubank, S. Englehardt, M. Juarez, A. Narayanan, and C. Diaz, "The web never forgets: Persistent tracking mechanisms in the wild," in *Proc. 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, Scottsdale, AZ, USA, 2014, pp. 674–689.